A SURVEY-BASED ANALYSIS OF NTFS AND FAT32 FILE SYSTEM PERFORMANCE

A MINI PROJECT - I REPORT

Submitted by

DHANSHIKA R (71812405011)

DHARSHINI S (71812405016)

LITHIGAA GJ (71812405051)

In partial fulfilment for the award of the

degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF INFORMATIONTECHNOLOGY

SRI RAMAKRISHNAENGINEERING COLLEGE,

COIMBATORE

ANNA UNIVERSITY: CHENNAI 600025

April 2026

SRI RAMAKRISHNA
ENGINEERING COLLEGE
[Educational Service: SNR Sons Charitable Trust]
Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.
www.srec.ac.in
+917530089996

ANNA UNIVERSITY: CHENNAI 600025

BONAFIDE CERTIFICATE

Certified that this project report entitled **"A Survey-Based Analysis of NTFS and FAT32 File System Performance"** is the bonafide work of **DHANSHIKA R (71812405011),DHARSHINI S(71812405016), LITHIGAA GJ(71812405051),** who carried out the **20IT276 - Mini Project I** under my supervision.

SIGNATURE
Dr. K.Deepa,
Professor
SUPERVISOR

Department of Information Technology
Sri Ramakrishna Engineering College
Coimbatore - 641022

SIGNATURE
Dr. N. Susila,
Professor and Head
HEAD OF THE DEPARTMENT

Department of Information Technology
Sri Ramakrishna Engineering College
Coimbatore – 641022

Submitted for Project Viva Voce Examination held on _____

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

We have immense pleasure in expressing our wholehearted thankfulness to **Dr. Sundar Ramakrishnan**, Managing Trustee, SNR Sons Charitable Trust and **Shri. S. Narendran**, Joint Managing Trustee, SNR Sons Charitable Trust for giving us the opportunity to study in our esteemed college and for very generously providing more than enough infrastructural facilities for us to get molded as a complete engineer.

We wish to express our profound and sincere gratitude to **Dr. A. Soundarrajan,** Principal, for inspiring us with his Engineering wisdom. We also wish to record our heartfelt thanks for motivating and guiding us to become Industry ready engineers with inter-disciplinary knowledge and skillset with her multifaceted personality.

We extend our sincere thanks to the overall coordinators, panel coordinators, panel heads, panel members for helping us in all our activities to develop confidence and skills required to meet the challenges of the industry for effectively deliver the project outcomes.

We also express our gratitude, sincere and indebted thankfulness thanks to **Dr. N. Susila,** Professor and Head, for giving us support and guidance to complete the project duly. We owe our deep gratitude to our project supervisor **Dr.K.Deepa,** Professor, Department of Information Technology who took keen interest in our project work and guided us all along with all the necessary inputs required to complete the project work.

We express our sincere thanks to our project coordinator **Mr.G.Ramsundar,** Assistant Professor (Sl. Gr.), the evaluators teaching faculty members and supporting staff members of the department for evaluating the project and providing valuable suggestions for improvements.

ABSTRACT

A Survey-Based Analysis of NTFS and FAT32 File System Performance

The role played by file systems in data storage and retrieval in current computing platforms is critical. Some of the popular file systems in use include FAT32 and NTFS file systems. This paper examines the analysis of the performance, reliability, fragmentation, and storage capabilities of FAT32 and NTFS file systems through an experiment. The experiment was carried out by partitioning storage devices to install FAT32 and NTFS file systems, and some file operations including file copy, file creation, and deletion were done. Performance aspects such as access time and fragmentation were measured using the internal utilities of the systems. The results show that FAT32 file system is efficient with small files since it has a simple structure, whereas NTFS performs better with large files because of its journaling capabilities. The results of this study reveal that NTFS is more reliable, efficient, and secure than FAT32. Nevertheless, FAT32 continues to be valuable in portable media because it works well in various operating systems. The purpose of this study was to identify the strengths and weaknesses of each file system and provide guidance on which file system should be used depending on the needs of the user.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	i
	LIST OF FIGURES	iv
	LIST OF ABBREVIATIONS	v
1	INTRODUCTION	1
2	LITERATURE REVIEW	7
3	PROJECT DESCRIPTION	13
	3.1 PROBLEM DEFINITION	13
	3.2 PROBLEM STATEMENT	14
	3.3 OBJECTIVES	14
	3.4 METHODOLOGY	15
4	DESIGN AND CODING	21
	4.1 FLOW DIAGRAM	21
	4.2 NTFS ARCHITECTURE	22
	4.3 FAT32 ARCHITECTURE	25
5	RESULTS AND DISCUSSION	31
	5.1 EXPERIMENTAL STUDY ANALYSIS	31
	5.2 DISCUSSION	39
6	CONCLUSION AND SCOPE FOR FUTURE WORK	40
	6.1 CONCLUSION	40

	6.2 SCOPE FOR FUTURE WORK	**41**
	REFERENCES	**42**
	SUSTAINABLE DEVELOPMENT GOALS (SDG) ADDRESSED	**44**
ANNEXURE I	**APPENDICES - STANDARDS**	**46**
ANNEXURE II	**BASE PAPER**	**49**
ANNEXURE III	**PROJECT ACHIEVEMENTS**	**60**
ANNEXURE IV	**PLAGIARISM REPORT**	**61**

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
3.4	Block Diagram for NTFS and FAT32	18
3.4.1	Partition Creation Module	18
3.4.2	File Generation and Storage Module	19
3.4.3	File Access Time Measurement Module	20
3.4.4	Fragmentation Analysis Module	20
3.4.5	Storage Utilization Analysis Module	21
3.4.6	Result Recording Module	22
4.1.1	Flow Diagram	23
4.2.1	NTFS Architecture	24
4.3.1	FAT32 Architecture	27
5.2.1	FAT32 File System Selection During Partition Formatting	29
5.2.2	NTFS File Format Partition	30
5.2.3	NTFS File Access Time Measurement	31
5.2.4	FAT32 File Access Time Measurement	31
5.2.5	NTFS Pre Fragmentation Analysis	32
5.2.6	NTFS Post Fragmentation Analysis	32
5.2.7	FAT32 Pre Fragmentation Analysis	33
5.2.8	FAT32 Post Fragmentation Analysis	33
5.2.9	Disk Optimization Status Display	34
5.2.10	NTFS File Security Display	35
5.2.11	FAT32 File Properties Display	36

LIST OF ABBREVIATIONS

ACL	**Access Control List**
API	**Application Programming Interface**
CPU	**Central Processing Unit**
DBR	**DOS Boot Record**
FAT	**File Allocation Table**
FAT32	**File Allocation Table 32**
FS	**File System**
FSINFO	**File System Information Sector**
GB	**Gigabyte**
GUI	**Graphical User Interface**
HDD	**Hard Disk Drive**
KB	**Kilobyte**
LCN	**Logical Cluster Number**
MB	**Megabyte**
MFT	**Master File Table**
NTFS	**New Technology File System**
OS	**Operating System**
RAM	**Random Access Memory**
SSD	**Solid State Drive**
USB	**Universal Serial Bus**

CHAPTER 1

INTRODUCTION

1.1 Overview of File Systems

The file system is considered one of the essential parts of any operating system, as it manages the storage, organization, and access of information on the storage device. The storage devices like hard disk drives (HDD), solid-state drives (SSD), and removable media need a way to store the files systematically to ensure that the user and software programs can retrieve the files easily. File system is used for such purposes. The file system in computer performs some crucial tasks within the computer system. These include creation, modification, deletion, and organization of files in an organized fashion. It also manages the allocation of space on the disk to store the file without overlapping any other file. Finally, the file system stores the metadata for each file, including the name of the file, size of the file, time of creation, date of modification, and permissions of the file.

In the course of time, there have been many changes in the file system. The first computers had only basic file systems since the storage capacity was very low, as well as the processing speed. With advances in storage technology and an increased need for massive data storage, more complex file systems have been created. Current file systems provide large storage space, better protection of stored information, fast file searching, and recovery methods that can restore files from a malfunctioning system.

There are various file systems for various operating systems because each of them has some specific features and goals. For instance, Microsoft Windows has FAT32, NTFS, and exFAT file systems. In addition, there is a variety of file systems that are used in Linux operating systems, namely, ext2, ext3, ext4, and XFS. Moreover, macOS has a new file system called APFS, and Unix-based systems use UFS.

File systems also have a great significance when it comes to the reliability of data. In case of any system failure or power outage, a highly sophisticated file system employs recovery procedures, such as journaling, that ensure data consistency. In turn, this guarantees data integrity and allows for a quick system restoration without causing any loss of data.

Also, the performance of the computer system depends greatly on file systems. The performance in terms of how quickly file creations, reading, updating, and deleting are carried out mostly depends on file systems used. Thus, choosing proper file systems is crucial from the standpoint of designing efficient computer systems.

1.2 File Systems in Windows Operating Systems

Windows operating systems come with various file systems suitable for handling varying storage needs. Among the file systems that are commonly found in Windows operating systems are FAT, FAT32, NTFS, and exFAT. Over the years, the file systems have undergone improvements in order to enhance their efficiency and increase storage capacity.

FAT (File Allocation Table) is an older file system that has been developed specifically for personal computers. It was common in early versions of Microsoft Windows and its predecessors because of its simplicity. The FAT file system divides the storage space into clusters and keeps track of file locations through a FAT. While the FAT file system was efficient enough in managing files, it had various weaknesses such as low storage capacity and security. I an effort to address the weaknesses of FAT versions, Microsoft developed the FAT32 as a more advanced version capable of handling larger storage devices. The file system operates using a 32-bit addressing method which enables it to manage bigger disks than FAT16. Currently, the FAT32 file system is widely used in removable

storage devices such as USB flash drives, memory cards, and external hard drives.

The New Technology File System (NTFS) is a modern file system created by Microsoft, offering enhanced capabilities such as increased reliability, security, and efficient storage. Some of the capabilities offered by NTFS include encryption, file permissions, compression, and journaling. This makes NTFS suitable for use in today's computer environment where security and reliability of data storage are important considerations.

One file system used in the Windows operating system is Extended File Allocation Table (exFAT). This particular file system is designed for flash drives and offers support for very large file sizes. In addition, exFAT is more compatible with the Windows and Mac OS X operating systems. In summary, there are many file systems available in Windows operating systems, each with different capabilities to meet various storage requirements. Two of the file systems commonly used are FAT32 and NTFS.

- **Introduction to NTFS File System**

NTFS or New Technology File System is an advanced file system designed by Microsoft in order to overcome the limitations of previous file systems, like FAT32. A critical part of NTFS is Master File Table, which contains a detailed information regarding each file or directory on the disk. Moreover, this advanced file system employs sophisticated allocation schemes which minimize fragmentation and increase the speed of the whole process.

Another characteristic of the NTFS is a number of its additional capabilities, like file permissions, encryption, compression, and journaling. File permissions make it possible to regulate who can access certain file. Furthermore, this file system uses encryption to protect sensitive information and compress information for better utilization of available space.Journaling allows the system to save information about file operations performed before executing them,

ensuring quick recovery from possible system crashes. As a result, NTFS is much more reliable than FAT32.Finally, the NTFS allows working with large files and disk partitions, which makes it appropriate for use in databases and similar applications.

- **Introduction to FAT32 File System**

FAT32 (File Allocation Table 32) is perhaps the most common type of file system. It appeared as an upgrade to previous FAT versions, as it was required to work with larger hard drives and to enhance performance. In FAT32, files are located on the hard drive according to information that is kept in the form of a table.

In FAT32, all the space on the hard drive is partitioned into clusters. Files are placed in clusters; the number of clusters per file depends on the size of the file itself. Information on what files are stored in which clusters is contained in the File Allocation Table. FAT32 is very simple and easily supports many operating systems.

The greatest advantage of FAT32 is its high level of compatibility. It can be used in various operating systems: Windows, Linux, and macOS.But there are also many disadvantages of the FAT32 file system. For instance, the biggest disadvantage is that the maximum limit for a file in the FAT32 file system is 4 GB. It is not possible to save files that have a size greater than this limit. Another disadvantage of the FAT32 file system is that it does not have any built-in security features like encryption. The FAT32 file system does not have any fragmentation problem.

1.3 File Systems in Linux Operating Systems

File systems in Linux operating systems include several different types intended to ensure flexibility, scalability, and performance. Common Linux file systems include such types as ext2, ext3, ext4, XFS, and Btrfs. These file systems allow handling of large volumes of storage space and efficient data processing.

One of the earliest file systems for Linux operating systems was ext2 (Second Extended File System). This type of file system allowed more effective management of the storage space than previous file systems, however, did not possess journaling. This means that system restoration could take much time after any abrupt shutdown. In order to address this issue, the ext3 (Third Extended File System) file system was created, allowing journaling. In other words, ext3 records all operations before executing them. This results in much better performance and reliability. ext3 has become one of the most common file systems because of its reliability and ext2 compatibility.

ext4 (Fourth Extended File System) offers larger file size support and improved performance. It implements advanced allocation strategy which reduces fragmentation and enhances storage space utilization. Ext4 is the most common file system today.XFS and Btrfs are some other sophisticated Linux file systems. XFS is a very efficient file system that works best in high-speed computing environments where speed and efficiency are critical. Btrfs offers advanced capabilities like snapshots, compression, and resilience. Linux file systems have gained recognition due to their flexibility and dependability. They offer large amounts of storage capacity and perform effectively in enterprise-scale applications.

1.4 File Systems in Mac OS and UNIX Operating Systems

The file systems vary from one operating system to another depending on its requirements. This means that there are different file systems for different

operating systems, which are created to serve particular purposes. For instance, Mac OS makes use of the Apple File System (APFS). It is meant to optimize performance on SSDs, and some of its features include support for encryption, snapshotting, and advanced storage allocation.

UNIX operating system uses UFS (Unix File System), which has been known for its reliable and steady performance, in addition to being a good file storage system. In mobile operating systems, the common file systems include ext4 and the Flash-Friendly File System (F2FS) for flash storage devices. Also, embedded systems use customized file systems. It can be observed that different storage methods exist in different operating systems.

CHAPTER 2

LITERATURE SURVEY

File systems are a fundamental component of modern computing, responsible for organizing, storing, and retrieving data efficiently on storage devices. They act as a structured interface between users, applications, and hardware, ensuring smooth data access and management. Designed to meet diverse requirements such as compatibility, performance, and security, different file systems are widely used across various domains. From general computing to industrial applications and digital forensics, file systems play a crucial role in enhancing system efficiency, reliability, and overall data integrity.

[1] M. Karresand, S. Axelsson, G. O. Dyrkolbotn – "Disk Cluster Allocation Behavior in Windows and NTFS"(2019)

The experiments revealed that allocation behavior varies based on factors like file writing method (block vs. stream), disk fragmentation, and availability of free space. In block writing, files are generally allocated in fragments of decreasing size, both within a file and across successive file operations; however, this does not always strictly adhere to the best-fit strategy, particularly when large contiguous free areas are present.

For stream writing, the allocation pattern differs significantly, with fragments typically starting very small and gradually increasing in size as more data is written. This occurs because the system does not know the final file size in advance and must allocate storage incrementally, demonstrating how allocation strategies dynamically adapt to writing behavior and system constraints.

The study highlights the importance of these allocation patterns in digital forensics, especially for tasks such as file carving and timestamp verification. Understanding how data is physically distributed on disk enables investigators to reconstruct fragmented files and infer operation sequences more accurately. The authors conclude that deeper knowledge of file system allocation behavior can enhance forensic efficiency and recommend further research across different systems to improve reliability and generalization of these findings.

[2] Li Qiyang , Zhang Quanxin, Tan Yu - an, Li Yuanzhang, Zheng Jun " Research Allocator Strategy of FAT32 File System Based on Linux & Windows" (2016)

The study conducted by Li Qiyang, Zhang Quanxin, Tan Yu-an, Li Yuanzhang, and Zheng Jun focuses on analyzing the allocator strategy used in the FAT32 file system in both Linux and Windows operating systems. According to the authors, FAT32 is widely used in storage devices such as USB flash drives, SD cards, digital cameras, and mobile devices due to its compatibility and efficiency. The research explains that as storage devices become more common, the amount of stored data increases rapidly, making data security and proper storage management more important. The authors emphasized that understanding file allocation strategies helps users protect data and improve storage performance

The study further explains the structure of FAT32 file systems by analyzing important sectors such as the DBR (DOS Boot Record) sector and the FSINFO sector. According to the authors, the DBR sector stores general information such as bytes per sector, sectors per cluster, and root directory details. The FSINFO sector records dynamic information such as the number of free clusters and the next available cluster location. The research highlights that an attribute called FSI_Nxt_ Free plays an important role in identifying the next free cluster during file allocation, helping maintain efficient storage operations.

The authors also analyzed allocator strategies used in Linux-based FAT32 systems. The research explains that Linux uses a next available allocation strategy, where the system starts searching for free clusters from the most recently allocated position. The study described how this strategy helps speed up file allocation and reduces repeated searching from the beginning of the storage area. This study method with Windows FAT32 allocation strategies and found that Windows uses a more complex allocation method rather than strictly following traditional allocation techniques.

Furthermore, the study conducted several experiments using SD cards with different storage capacities and cluster sizes to verify allocation behavior. The authors found that FAT32 systems organize clusters into logical units called blocks, each containing 65,536 clusters. The research concluded that Windows FAT32 allocation attempts to maintain continuous storage whenever possible to reduce fragmentation and improve performance. The study highlighted that understanding FAT32 allocation strategies helps improve file recovery processes and enhances overall data security.

[3] N. R. Gosavi, S. S. Morade, D. D. Khartad – "Designing USB based FAT32 file systemfor CNC Machine Using Dual Core DSP Processor" (2017)

In this project, the authors propose a USB-based FAT32 file system integrated with a dual-core DSP processor to improve data transfer and processing efficiency in CNC machines. The study highlights the limitations of traditional RS232 communication, particularly its low speed (115.2 Kbps), and demonstrates how replacing it with USB 2.0 communication significantly enhances performance up to 12 Mbps. The authors further design and implement a FAT32 file management system that supports essential file operations such as read, write, copy, and conversion of toolpath files into machine-compatible formats. Experimental results show that the proposed system reduces tool processing time and improves overall system efficiency. Additionally, the use of the TMS320F28377D dual-core DSP processor enables real-time processing and optimized computation for CNC operations.

The study also analyzes file conversion performance, showing that processing time increases with file size but remains efficient due to optimized algorithms. Moreover, software optimization techniques significantly reduce memory usage, enhancing system performance.

Moreover, software optimization techniques significantly reduce memory usage, enhancing system performance. The findings emphasize the importance of high-speed communication interfaces and efficient file systems in modern CNC applications, suggesting that integrating USB-based architectures with embedded processing can greatly improve automation and reliability in industrial systems.

[4] Raksha Rane, Asmit Singh – "Demystifying File Systems: Comprehensive Exploration of Data Organization" (2024)

The study conducted by Raksha Rane and Asmit Singh focuses on understanding the role of file systems in organizing and managing data within operating systems. According to the authors, file systems act as the foundation for storing, retrieving, and managing data efficiently on storage devices such as USB drives and solid-state drives. The research explains that file systems provide a structured method for organizing files and directories, ensuring proper communication between users, applications, and storage hardware. The authors emphasized that without a file system, storage devices would not be able to manage data effectively, making file systems essential for modern computing environments.

The study further explains the importance of performance and speed in file systems, highlighting how faster read and write operations improve overall system efficiency. According to the authors, performance plays a major role in applications that require quick data access, especially in multitasking and server environments.

The authors also performed a comparative study of multiple file systems, including FAT32, NTFS, exFAT, and APFS. The research explains that FAT32 is widely used due to its compatibility across different operating systems, while NTFS provides advanced features such as encryption, file compression, and improved security mechanisms. The study described exFAT as suitable for large storage devices due to its support for large file sizes, while APFS was developed specifically for Apple devices with features like copy-on-write and snapshots. This analysis helped identify the strengths and limitations of each file system based on compatibility, storage capacity, and security features.

Furthermore, the study conducted performance benchmarking tests to evaluate the efficiency of different file systems using real-time experiments. According to the authors, the tests measured read and write speeds using benchmark tools on SSD storage devices. The results showed that NTFS and exFAT provide better performance improvements.

[5] Liu Naiqi, Wang Zhongshan, Hao Yujie, Qin Ke – "Computer Forensics Research and Implementation Based on NTFS File System" (2008)

The study conducted by Liu Naiqi, Wang Zhongshan, Hao Yujie, and Qin Ke focuses on the analysis of the NTFS file system and its importance in computer forensics. According to the authors, computer forensics involves collecting, preserving, and analyzing digital evidence without modifying the original data. The research explains that digital evidence such as documents, images, and system logs must be handled carefully to maintain data accuracy. The authors emphasized that the NTFS file system is widely used in Windows operating systems and plays a major role in storing and managing digital.

The study further explains the internal structure of the NTFS file system, highlighting the importance of the Master File Table (MFT). According to the authors, the MFT acts as the central storage location that contains information about all files and directories. Each file and directory has a unique entry in the MFT, which stores metadata such as file name, file size, creation time, and storage location. The researchers described how NTFS uses different attributes to store file details and how resident and non-resident attributes help manage both small and large files effectively.

The authors also proposed an algorithm for reconstructing directory tree from deleted files and directories. The research explains that deleted files can be recovered by scanning MFT entries and identifying the parent directory references. The study introduced a well-structured binary tree method to organize deleted file information, which improves the efficiency of directory reconstruction. The researchers highlighted that scanning both continuous and fragmented MFT tables is necessary to recover complete directory structures accurately.

Furthermore, the study analyzed the storage principle of Data Runs used in NTFS to manage file data stored across disk clusters. This study analysis the performance of a newly developed recovery software called SmoothRecovery with an existing tool known as EasyRecovery. The results showed that SmoothRecovery provided better performance in identifying and restoring deleted files. The study concluded that understanding NTFS internal structure and recovery methods significantly improves the accuracy and efficiency of digital forensic investigations.

CHAPTER 3

PROJECT DESCRIPTION

3.1 PROBLEM IDENTIFICATION

In today's computing environment, there is a lot of data being created and stored in different storage media such as HDD, SSD, or external storage devices. The performance of storing and retrieving the data greatly depends on the file system being used. File systems determine how fast the data will be accessible, the efficiency of space utilization, and security of files in terms of accessing them.FAT32 and NTFS are two types of file systems widely used in Windows operation system. FAT32 is often adopted by many portable media due to its efficiency and ability to be used across various operating systems. Nevertheless, there are many disadvantages associated with this type of file system, such as limitation in file size, absence of security features, and higher fragmentation level while working with large data. Unlike FAT32, NTFS is created to ensure faster performance and greater reliability and security. This file system incorporates some advanced features like journaling, file permissions, encryption, and metadata management. Even though this file system has advanced functionality, it is less efficient in terms of using system resources. For these reasons, the choice of the right file system has become a major issue for both system administrators and users. But many users may be not fully informed about differences between FAT32 and NTFS in relation to performance, storage management, and reliability. In this regard, it is necessary to investigate these file systems with respect to their performance and effectiveness.

This project will fulfill the mentioned need since it involves a survey-based and experimental research of FAT32 and NTFS file systems. It will evaluate the performance of the mentioned systems via performing real-time file actions like formatting, copying, determining access time, etc.

3.2 PROBLEM STATEMENT

Traditional file systems such as FAT32, despite their simplicity and wide compatibility, are not well-suited for modern computing environments due to their lack of security, fault tolerance, efficient storage management, and scalability. As data volumes increase and systems require reliable, secure, and high-performance storage solutions, these limitations become more critical. FAT32 is unable to effectively handle large files, multi-user access, and system failures, leading to potential data loss and reduced performance. Although advanced file systems like NTFS offer improved features such as structured metadata management, access control, efficient space allocation, and journaling.

3.3 OBJECTIVES

The primary goal of this project is to carry out an analysis of the FAT32 and NTFS file systems under different storage activities.The goals of this project include the following:

- Understanding the working principle of FAT32 and NTFS file systems.
- Partitioning of storage devices using FAT32 and NTFS file systems.
- File access time measurement with the help of command prompt.
- Analyzing the performance of file copies in both file systems.
- Study of storage allocation processes in FAT32 and NTFS.
- Fragmentation studies during file operations.
- Storage efficiency study in FAT32 and NTFS file systems.
- Advantages and disadvantages of FAT32 and NTFS file systems.
- Applicability of FAT32 and NTFS file systems in different storage environments.

3.4 METHODOLOGY

The steps undertaken in this developed system are systematic and structured to the performance, storage factors and levels of security of FAT32 and NTFS file systems. This methodology involves experimental setup of directories partitions and files, triggering file operations, measuring amount of memory access to retrieve data for restructuring on runtime or in storage level fragmentation.

The System Structure consists of following Modules:

- Partition Creation Module
- File System Formatting Module
- File Creation and Storage Module
- File Operation Execution Module
- File Access Time Measurement Module
- Fragmentation Analysis Module
- Security and Permission Analysis Module
- Storage Utilization Analysis Module
- Result Recording Module
- Performance Module

These modules combine to form an integrated system for accurate observation of file system behavior. These steps are integrated to provide reliable file system performance evaluation between FAT32 and NTFS file systems.

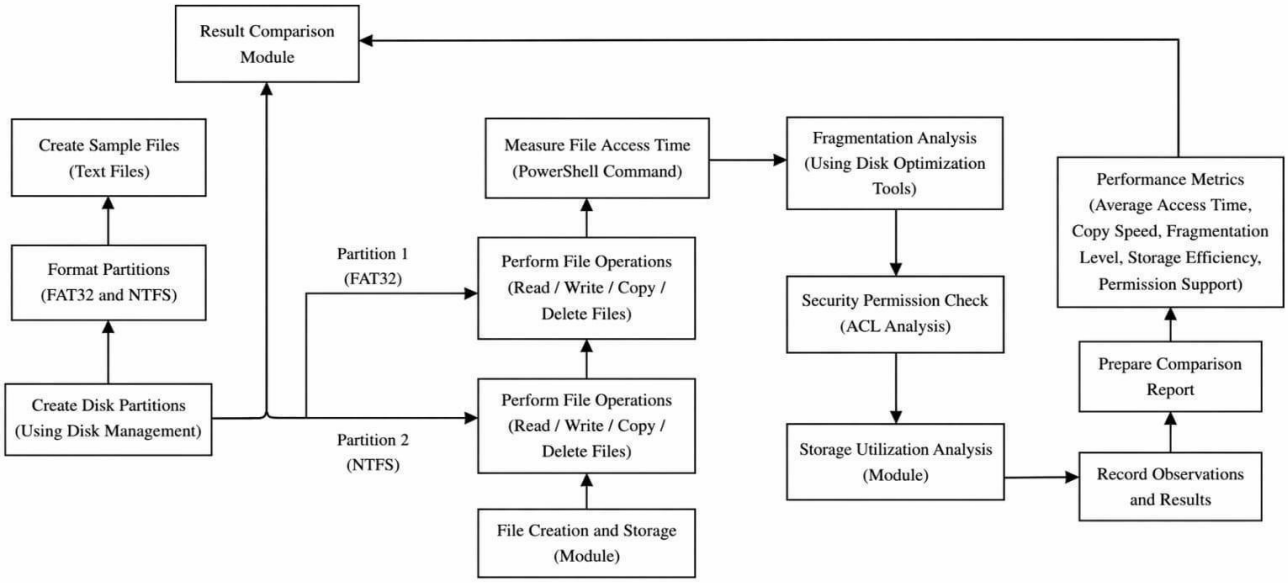


Fig 3.4 Block Diagram for NTFS and FAT32

3.4.1 Partition Creation Module

The first phase of this research method consists of making two distinct partitions within the storage medium in order to conduct FAT32 and NTFS file systems tests. These partitions were made through Windows Disk Management software. The use of two partitions within one storage medium provides uniformity in the hardware components of the study.

The first step includes locating available space within the storage medium, then splitting this space into two equal partitions. Both partitions have been named separately in order to distinguish between the FAT32 and NTFS file systems.

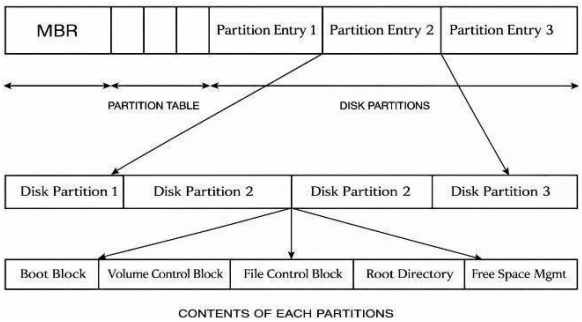


Fig 3.4.1 Partition Creation Module

3.4.2 File System Formatting Module

Following the completion of the partitioning process, the next process involved formatting the partitions into the FAT32 and NTFS file systems. The

formatting process sets out how the data will be arranged and managed on the storage media. While one partition was formatted using the FAT32 file system, the other partition was formatted using the NTFS file system using the Windows Disk Management utility.

The formatting process involved setting up various parameters such as the type of file system, allocation unit size, and volume label. Formatting into the FAT32 file system involves creating a File Allocation Table (FAT) for data management, whereas formatting into the NTFS file system involves creating additional structures like the Master File Table (MFT).

3.4.3 Files Generation and Storage Module

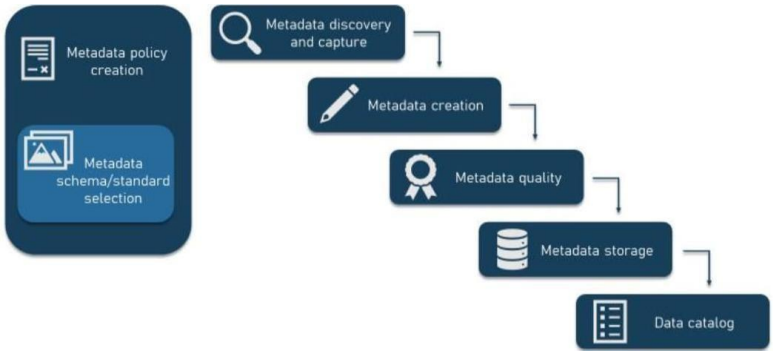


Fig 3.4.2 Files Generation and Storage Module

The purpose of this module is to create samples of files on FAT32 and NTFS partitions. Text files were created from command prompt commands to represent normal file use. The same files were created in both partitions to make sure that there will be the same size and content in each partition.

Creating files assists in analyzing how the clusters in FAT32 and NTFS are allocated. Many files were created to analyze the storage pattern and cluster allocation. The number of files and their sizes were noted above.

3.4.4 Module for File Operations Execution

Once sample files had been created, a number of operations were carried out on the FAT32 and NTFS file systems for the assessment of performance during operation. These include file copying, reading, writing, modifications, and deletions.

Copying was done within the same partition to test the performance in terms of duplicating data. Reading tests were used to check the speed at which data

could be retrieved, while writing tested the process of modifying files and saving them. Deletion operations involve deleting data files; FAT32 partitions remove data references from the allocation table, while NTFS partitions delete the files from the Master File Table (MFT).

3.4.5 File Access Time Measurement Module

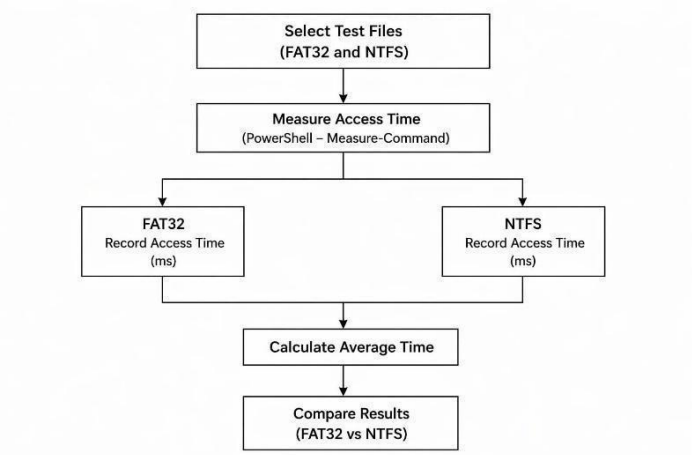


Fig 3.4.3 File Access Time Measurement Module

File Access Time Measurement Module Design Structure shows the method of determining the file access time in FAT32 and NTFS file systems. First, the files under analysis are chosen from the FAT32 and NTFS file systems. Using PowerShell commands such as Measure-Command, the file access time is measured several times to ensure accuracy in obtaining the results, and then the average file access time is determined.

3.4.6 Fragmentation Analysis Module

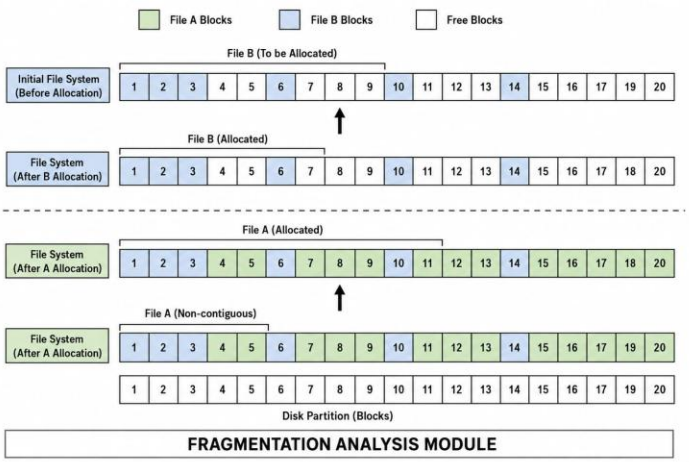


Fig 3.4.4 Fragmentation Analysis Module

The above figure explains fragmentation of disk partitions. Prior to fragmentation, the blocks are kept in contiguous memory. However, after performing many operations on files, they become spread out into various positions, causing fragmentation of the memory blocks. NTFS is generally less fragmented than FAT32 because of its efficient block allocation system.

3.4.7 Security and Permission Analysis Module

This module tests security functions present in FAT32 and NTFS file systems. The security permission analysis was performed with the help of file properties in Windows. This was achieved by accessing the Security tab.

The NTFS file system includes Access Control List (ACL). It enables the administrators to set permissions such as read, write and modify permissions. FAT32 file system does not have ACL support; hence, it is relatively less secure than NTFS file system. Security testing was done on the basis of the permissions set for files stored in NTFS partitioned drives. The presence of security functionality was documented.

3.4.8 Storage Utilization Analysis Module

Storage utilization analysis involves measuring the efficiency of disk usage by individual file systems. Disk features were analyzed in order to obtain used space and free space values.

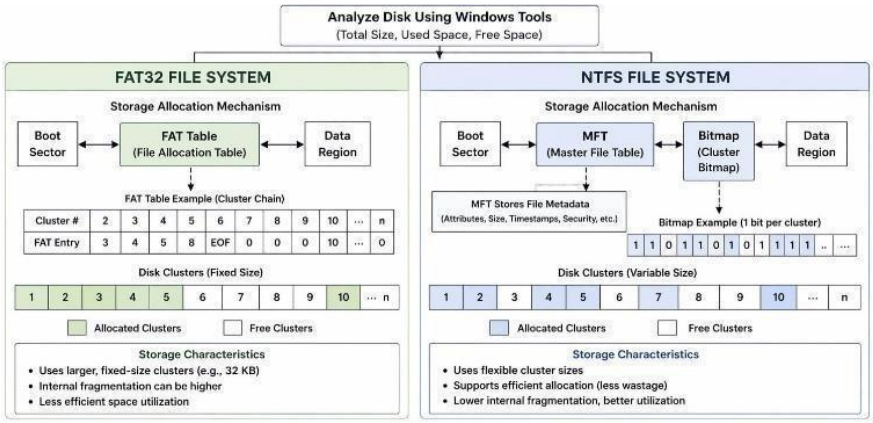


Fig 3.4.5 Storage Utilization Analysis Module

Allocation unit size affects the efficiency of storage. FAT32 may lose storage efficiency because of larger clusters. However, NTFS allows efficient storage allocation using flexible clusters. Results of storage utilization analysis were

documented for both partitions. This will help in identifying the most efficient file system concerning storage usage.

3.4.9 Result Recording Module

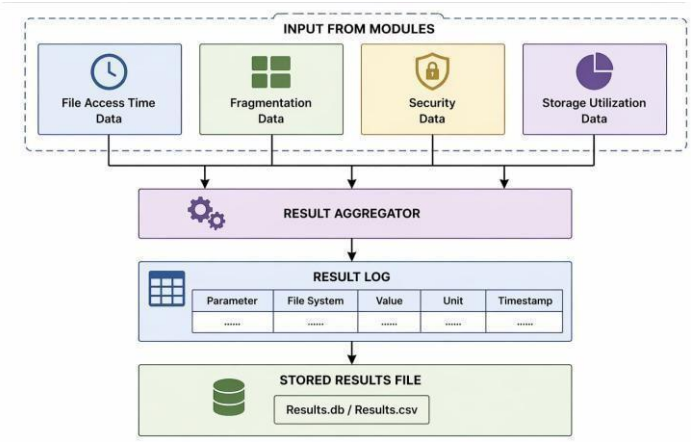


Fig 3.4.6 Result Recording Module

Results gathered through different modules were recorded appropriately. Such data include file access time, level of fragmentation, security availability, and storage utilization, among others.

3.4.10 Performance Module

In the case of performance evaluation, the emphasis lies on the analysis of gathered information regarding the FAT32 and NTFS file systems. The information was analyzed according to certain criteria such as the time taken for accessing files, their degree of fragmentation, usage of space, and other security issues. The obtained information has been presented through graphs for better comprehension.

CHAPTER 4

DESIGN AND CODING

4.1 PROCESS FLOW REPRESENTATION

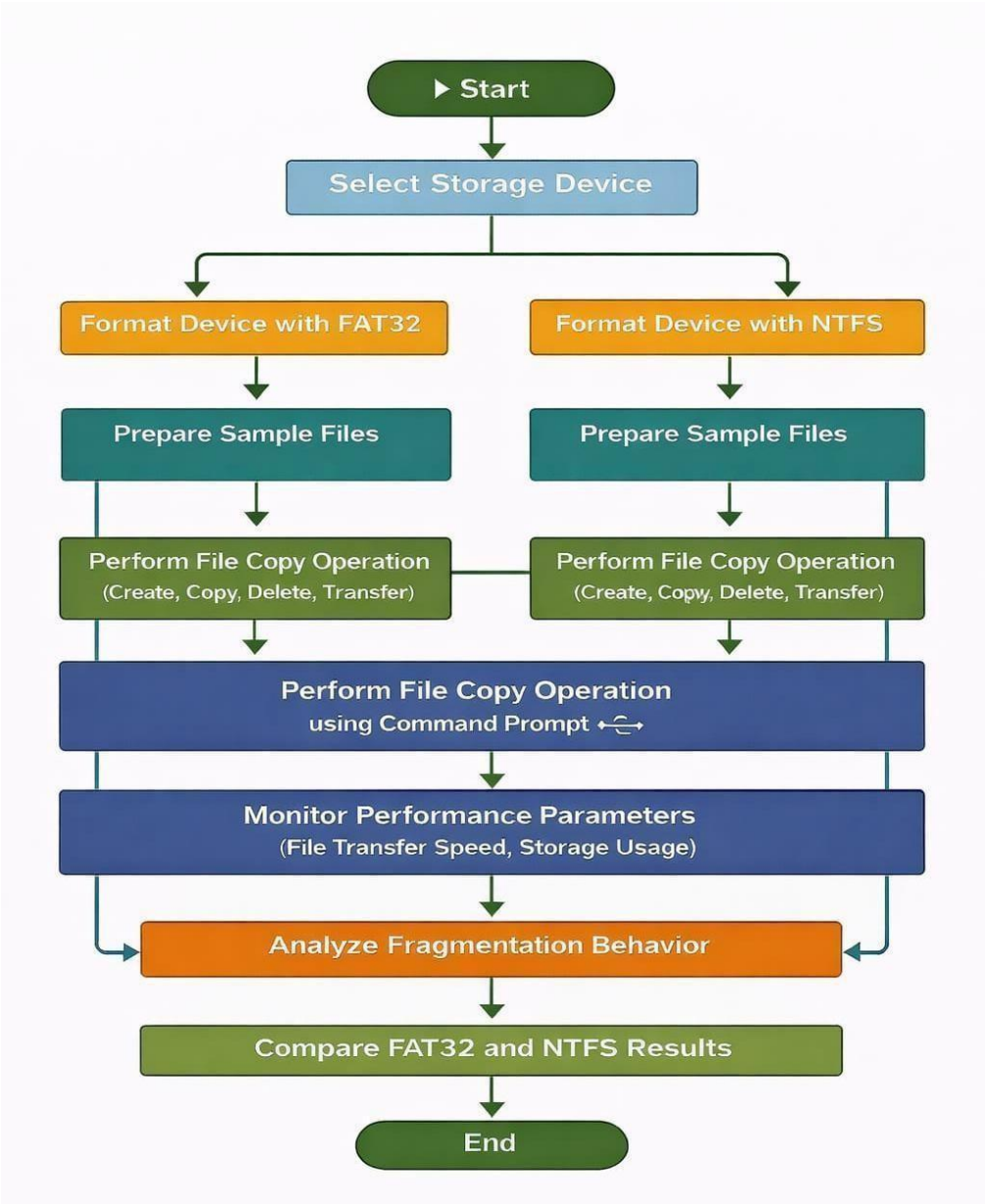


Fig 4.1.1 Flow Diagram

The flow chart shows the order of events that occur in the process of analysis of the two FAT32 and NTFS file systems. The procedure starts with the creation and formatting of disk partitions using the two file systems. Sample files are created and different file activities, like read, write, copy and delete are performed on them. Time is measured for accessing the files and analysis conducted on fragmentation, security, and space usage issues.

4.2 NTFS Architecture

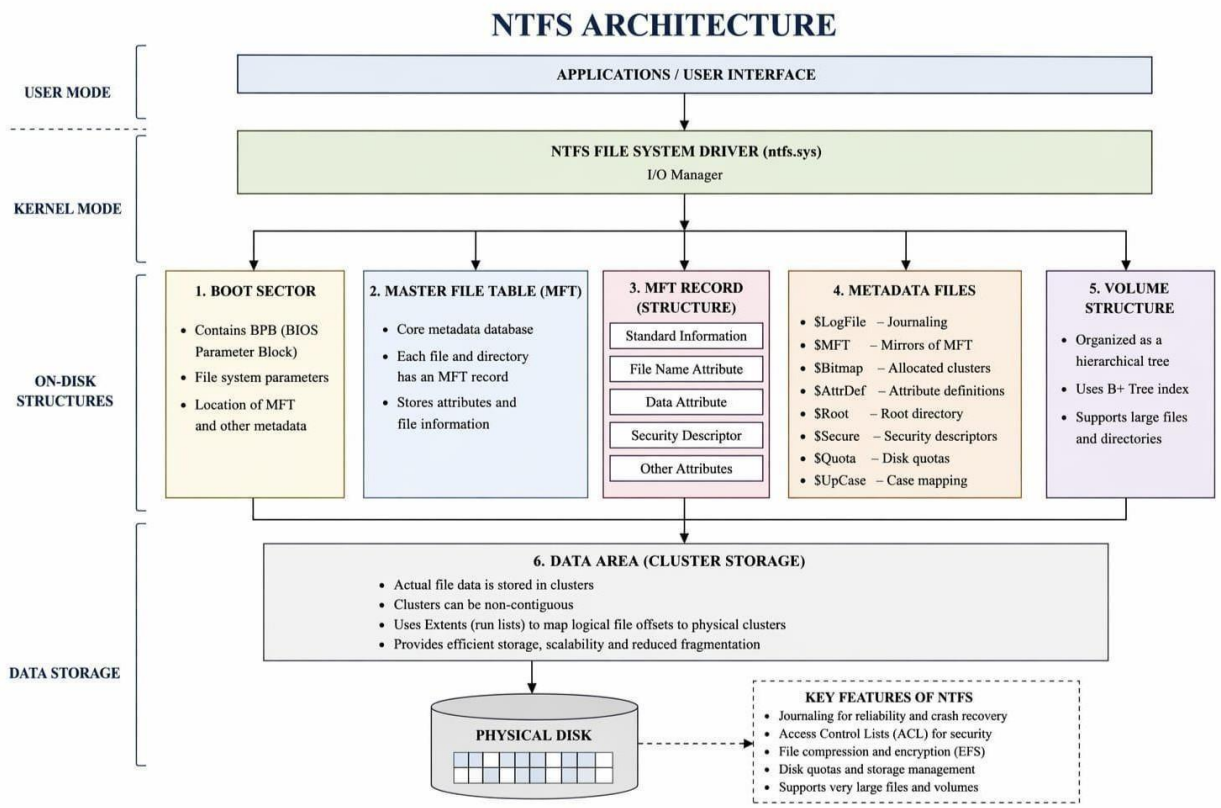


Fig 4.2.1 NTFS Architecture

The NTFS fragmentation architecture describes how file data is stored, managed, and sorted by the NTFS file system. Fragmentation happens when file data is stored in different non-continuous clusters in the storage disk. The NTFS file system manages fragmentations using its internal components including Journaling, Bitmap, Master File Table (MFT), and Access Control List (ACL).

1. Before Defragmentation (Fragmented)

In pre-defragmentation, file data is stored in different disk clusters. Whenever file creation, modification, and deletion take place in an NTFS environment, the operating system assigns any available free clusters. This creates fragmentation whereby fragments of the same file can be located at different positions.

- File data is scattered on various cluster spaces
- Disk reading/writing is relatively slow
- More movements occur in the disk

- System operation slows down
- Space utilization is inefficient
- Conditions before defragmentation show the effect of continuous file operations.

4.2.2 After Defragmentation (Non-Fragmented)

After defragmentation, fragmented files are arranged in contiguous clusters. File data arrangement takes place, and all fragments of a file are placed next to each other in the disk.

- Files are stored in sequential clusters
- Faster file access speeds
- Minimal disk movement
- System operations improve
- Efficient space utilization

4.2.3 Journaling ($LogFile)

Journaling is an important element of NTFS technology which assures reliable operation of data processing. $LogFile file keeps transactional records for all metadata changes which could be restored in case of failure.

- Keeps records of metadata modifications
- Facilitates safe re-allocation of clusters
- Provides integrity of data
- Prevents data loss and corruption
- Journaling increases reliability during defragmentation.

4.2.4. Bitmap ($Bitmap)

Bitmap is a special NTFS file that manages storage of clusters on the disk by recording their allocation status. Every bit indicates a state of particular cluster: free/allocated.

- Indicates cluster usage

- Facilitates free clusters locating
- Provides fast and efficient cluster allocation
- Minimizes chances for fragmentation
- Bitmap management provides effective organization of storage space.

4.2.5 Master File Table (MFT)

Master File Table (MFT) is the main NTFS database that holds information about all files and directories. It provides detailed information about each file and where its clusters are located on the disk. Role in Fragmentation:

- Provides mapping between data files and corresponding clusters
- Uses data runs (extents) to determine physical locations of file components
- Provides location updates during defragmentation
- Maintains file integrity
- File is fragmented when there is more than one extent used for file content.

4.2.6 Access Control List (ACL)

Access control lists (ACL) offer security permissions to files and folders in the NTFS system. ACL entries specify user rights on files and folders to read, write, or modify their content.

- Provides security permissions while moving files
- Guarantees that no changes are made to file access rights
- Documents security permissions changes via journaling feature
- Enables secure defragmentation activities
- ACL guarantees security permissions even if the location of files is changed during defragmentation.

4.2.7 Impacts of Fragmentation

- Slower file access
- Inefficient operation
- More disk movements

- Wearing out hardware

4.3 FAT32 ARCHITECTURE

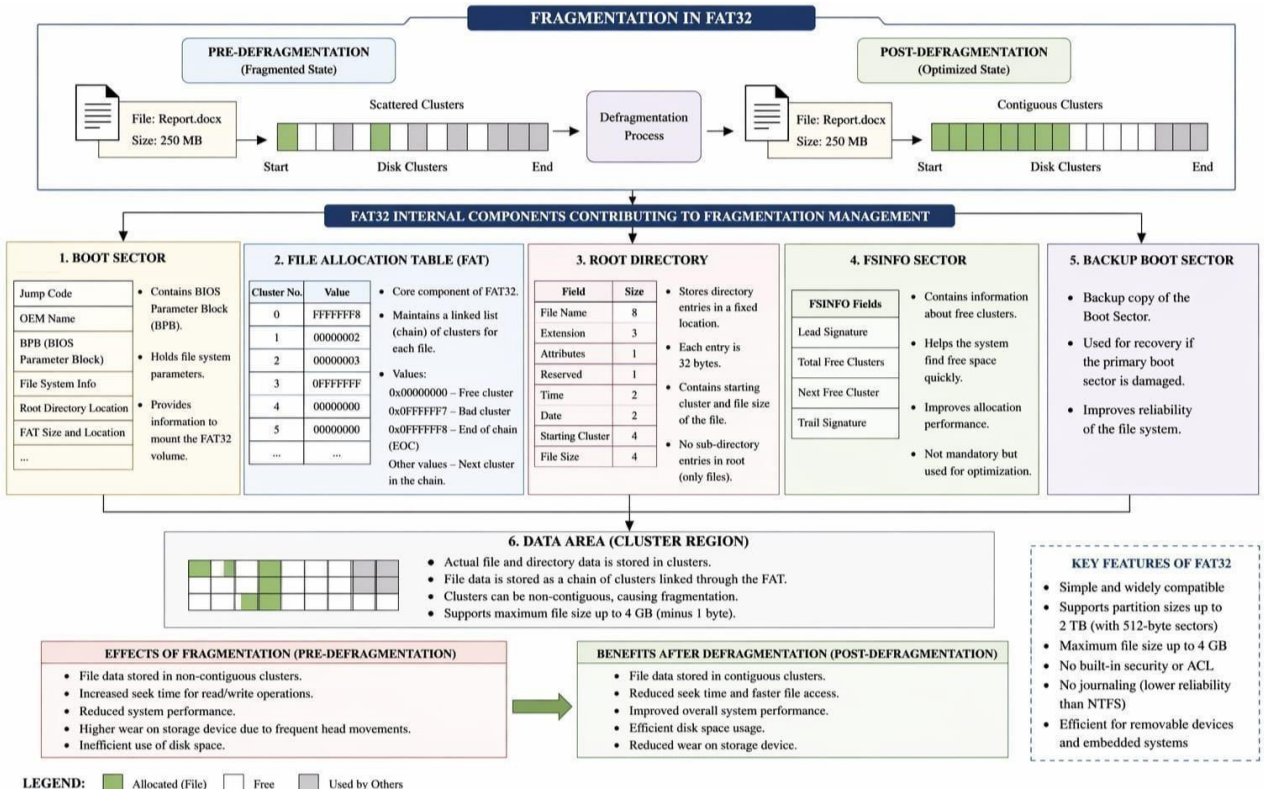


Fig 4.3.1 FAT32 Architecture

The model explains how files get stored, allocated, and managed in the FAT32 file system through file fragmentation. The file is fragmented because its data gets stored in dispersed clusters due to the many file creations, deletions, and modifications operations in the system. FAT32 file management uses the following system components; Boot Sector, File Allocation Table (FAT), Root Directory, and Data Area to store data.

1. Before Defragmentation (Fragmented)

When the system is not fragmented, files get stored in many clusters on the disk. Since the FAT32 allocates free sequential clusters and when no continuous clusters are found then files are allocated to separated clusters leading to fragmentation.

- File data is stored in many separate clusters.
- Operations involving read/write take long time
- Disk heads move many times
- The computer performs slow
- Storage efficiency is poor
- File operations increase file fragmentation.

4.3.2 After Defragmentation (Optimized)

After files are defragmented, they become contigent and thus can be accessed in one location. Files segments get relocated from their scattered positions into adjacent locations.

- Files segments are in continuous clusters
- Disk movement is minimized

4.3.3 Boot Sector

The Boot Sector contains various parameters of the system that are needed to start the initialization process of the FAT32 file system. The parameters include BIOS parameter block, which carries such details as cluster size, file system type and other properties.

- Gives information about disk organization
- Determines the position of FAT and data areas
- Aids in performing correct file allocations
- The Boot Sector provides correct identification of file system components.

4.4.4 File Allocation Table (FAT)

The File Allocation Table (FAT) is the component that controls the process of files storage in the FAT32 file system. Each FAT entry indicates cluster links forming the file chain.

- Allocates clusters
- Establishes connections between cluster
- Indicates next clusters in the chain
- Allows storing fragments of file in multiple clusters.

4.4.5 Root Directory

Root Directory stores details about the file and directories contained on the FAT32 volume. Directory entries contain file names, attributes, cluster numbers at which files starts and file sizes.

- Functionality in Fragmentation:

- Contains file location information
- Indicates starting clusters of each file
- Enables locating file clusters
- Root Directory enables locating of fragmented file portions.

4.4.6 FSINFO Sector

This sector provides free cluster space information to allow fast searching for the space that is free without analyzing the entire FAT table.

- It provides free cluster space information
- Improves allocation speed
- Reduces cluster search time
- This increases the efficiency of the allocation.

4.4.7 Backup Boot Sector

This sector contains a backup copy of the primary boot sector. The backup is used to recover any system information if the primary boot sector becomes corrupted.

- Backup sector facilitates data recovery
- Increases data reliability
- Makes sure that the data is safe on the disk
- This makes sure that the operation is reliable when writing information to the disk.

4.4.8 Data Area (Cluster Area)

Data area contains the actual files stored in clusters on the disk. This storage involves storing files using chain clusters. Clusters are connected using the FAT table. Fragmentation takes place in this area when there is no more consecutive clusters available.

4.4 CODING

Create Directories for Testing

```
mkdir D:\NTFS_Test
mkdir E:\FAT32_Test
mkdir C:\ACL_Test
mkdir C:\MFT_Test
```

Create Sample Files Using Command Prompt

```
echo Test file 1 > D:\NTFS_Test\file1.txt
echo Test file 2 > D:\NTFS_Test\file2.txt
echo Test file 3 > D:\NTFS_Test\file3.txt

echo Test file 1 > E:\FAT32_Test\file1.txt
echo Test file 2 > E:\FAT32_Test\file2.txt
echo Test file 3 > E:\FAT32_Test\file3.txt
```

Create Multiple Files Automatically

```
1..100 | ForEach-Object {
New-Item D:\NTFS_Test\file$_.txt -ItemType File
}
1..100 | ForEach-Object {
New-Item E:\FAT32_Test\file$_.txt -ItemType File
}
```

Display Directory Contents

```
dir D:\NTFS_Test
dir E:\FAT32_Test
dir C:\MFT_Test
```

Copy Large Number of System Files

```
xcopy C:\Windows\System32\*.exe D:\NTFS_Test\ /I
xcopy C:\Windows\System32\*.exe E:\FAT32_Test\ /I
```

Measure File Access Time

Measure-Command { Get-Item D:\NTFS_Test\file1.txt }

Measure-Command { Get-Item E:\FAT32_Test\file1.txt }

Generate Additional Files for Performance Testing

1..1000 | ForEach-Object {

New-Item C:\MFT_Test\file$_.txt -ItemType File

}

Retrieve NTFS File System Information

fsutil fsinfo ntfsinfo C:

View File System Information

fsutil fsinfo volumeinfo C:

Fragmentation Analysis (Pre-Optimization)

defrag D: /A /V

defrag E: /A /V

Perform Defragmentation

defrag D: /V

Batch file

defrag E: /V

Post-Defragmentation Analysis

defrag C: /A /V

Check Disk Optimization Status

defrag /C /A

Set NTFS Access Permissions (ACL)

icacls C:\ACL_Test /deny Guest:R

Verify ACL Permissions

icacls C:\ACL_Test

Display Disk Information

fsutil fsinfo drives

Delete Files (Optional Testing)

del D:\NTFS_Test\file1.txt

del E:\FAT32_Test\file1.txt

CHAPTER 5

RESULTS AND DISCUSSION

5.1 EXPERIMENTAL AND STUDY ANALYSIS

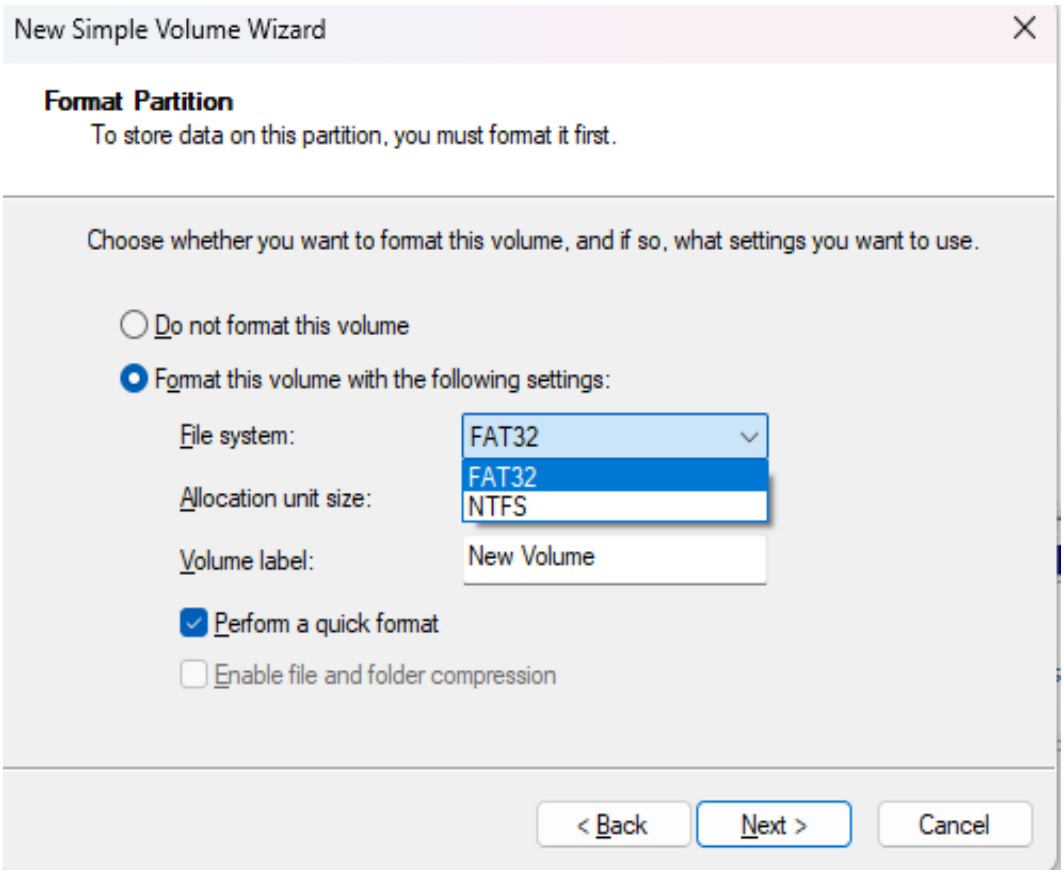


Fig 5.2.1: FAT32 File System Selection During Partition Formatting

Fig 5.2.1 shows the selection of the FAT32 file system during the partition formatting process using the New Simple Volume Wizard. The FAT32 option is selected from the file system list to conFigthe storage partition. This step prepares the storage volume for performing experimental analysis related to FAT32 file.

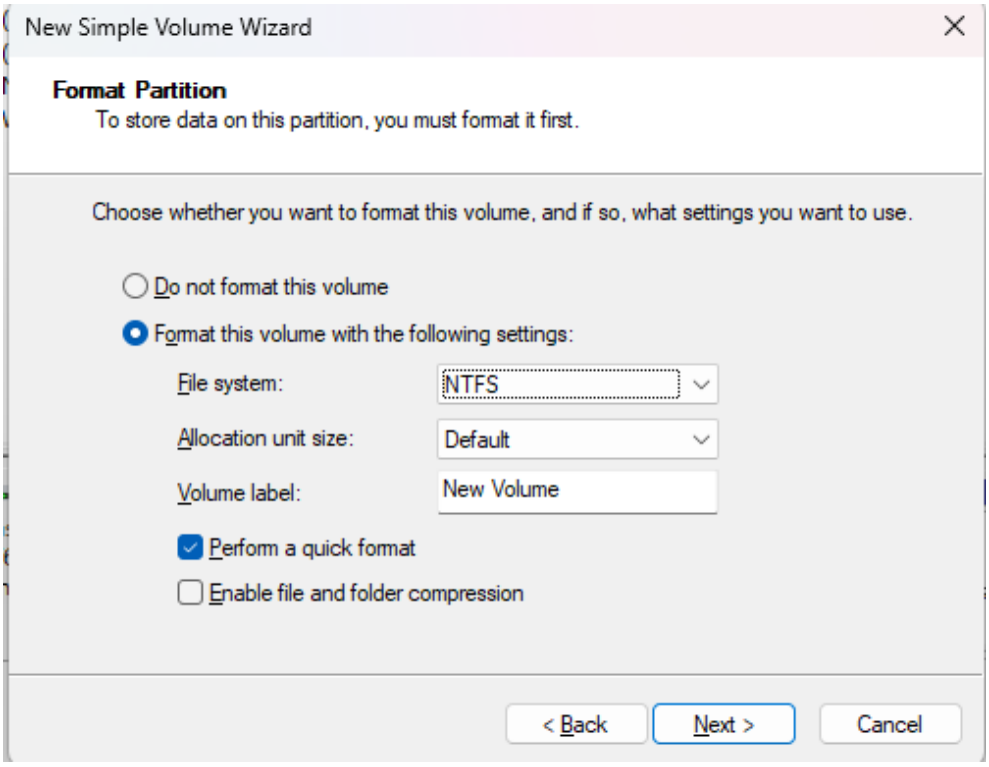


Fig 5.2.2: NTFS File Format Partition

Fig 5.2.2 The image shows the Windows “Format Partition” step where a volume is formatted using NTFS with default settings and quick format enabled.

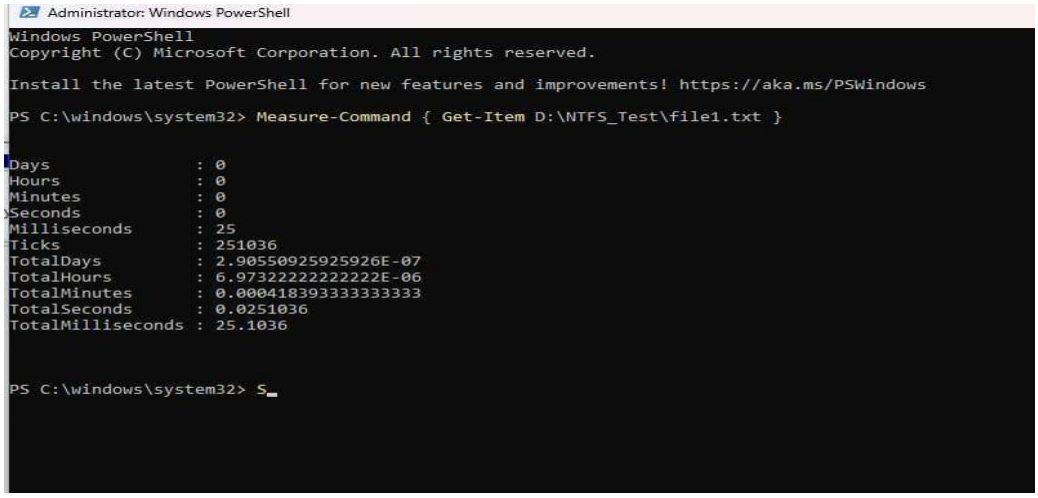


Fig 5.2.3: NTFS File Access Time Measurement Using PowerShell

Fig 5.2.3 shows the measurement of file access time in the NTFS file system using the PowerShell Measure-Command utility. The command calculates the time required to retrieve a file from the NTFS partition. This step is useful in evaluating the file retrieval speed of NTFS.

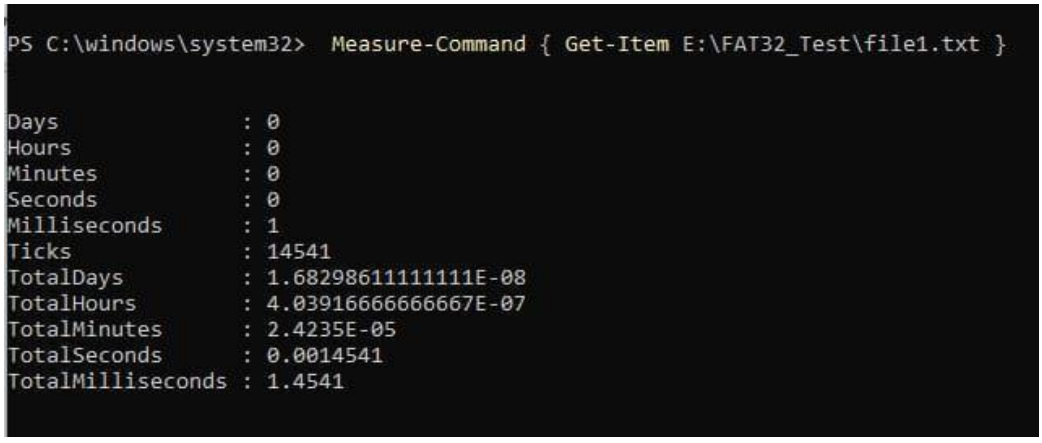


Fig 5.2.4: FAT32 File Access Time Measurement Using PowerShell

Fig 5.2.4 illustrates the measurement of file access time in the FAT32 partition. The execution output displays the time taken to access a file stored in FAT32. These values are later used for analyzing FAT32 and NTFS performance.

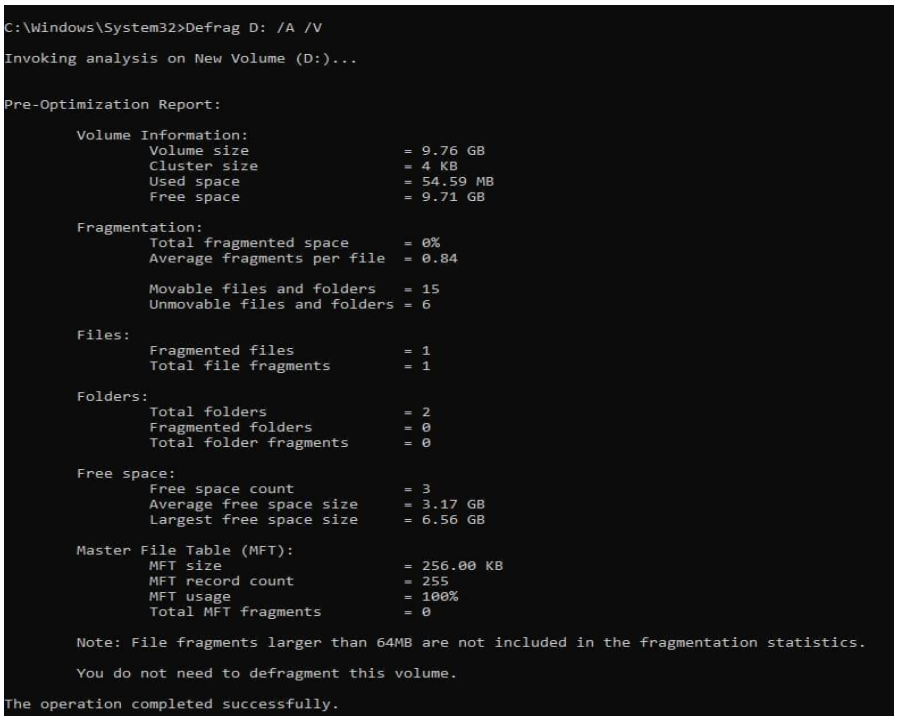


Fig 5.2.5 NTFS Per Fragmentation Analysis

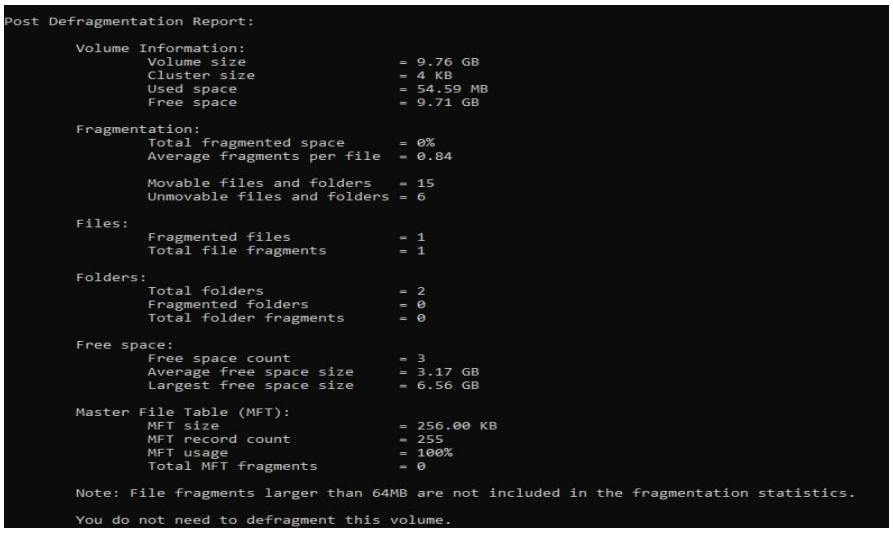


Fig 5.2.6 NTFS Per Fragmentation Analysis

Fig 5.2.5 & 5.2.6 the disk fragmentation analysis report generated for the NTFS partition. The report includes information such as volume size, cluster size, fragmented files, and Master File Table (MFT) usage. The NTFS file system shows minimal fragmentation due to its structured metadata management.

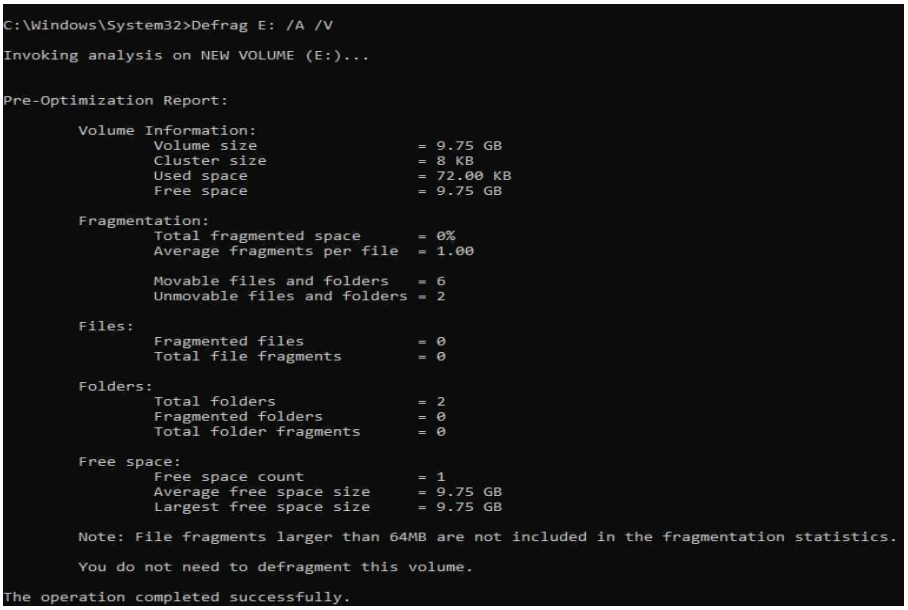


Fig 5.2.7: FAT32 Pre Fragmentation Analysis

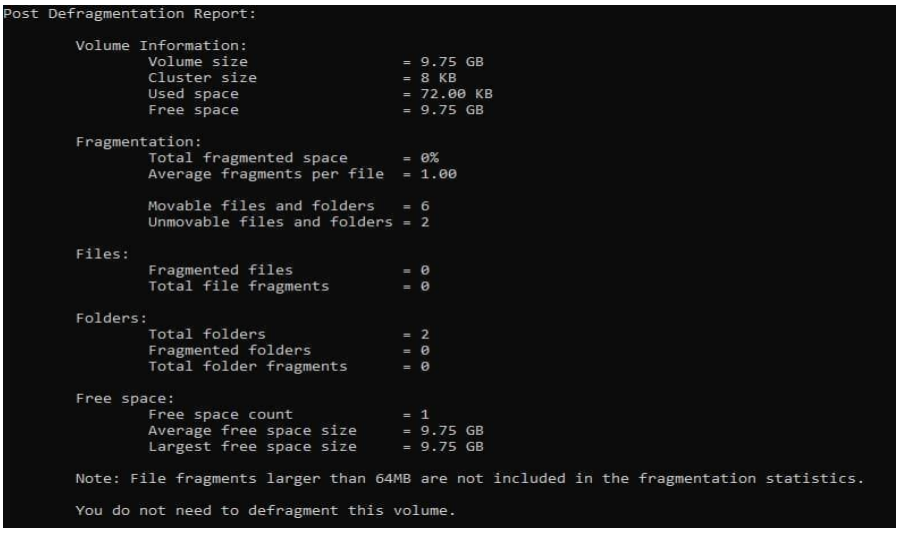


Fig 5.2.8: FAT32 Post Fragmentation Analysis

Fig 5.2.7 & 5.2.8 illustrates the fragmentation analysis results obtained from the FAT32 partition. The report displays cluster allocation statistics and fragmented file details, NTFS, FAT32 shows relatively higher fragmentation levels due to its simpler file allocation method.

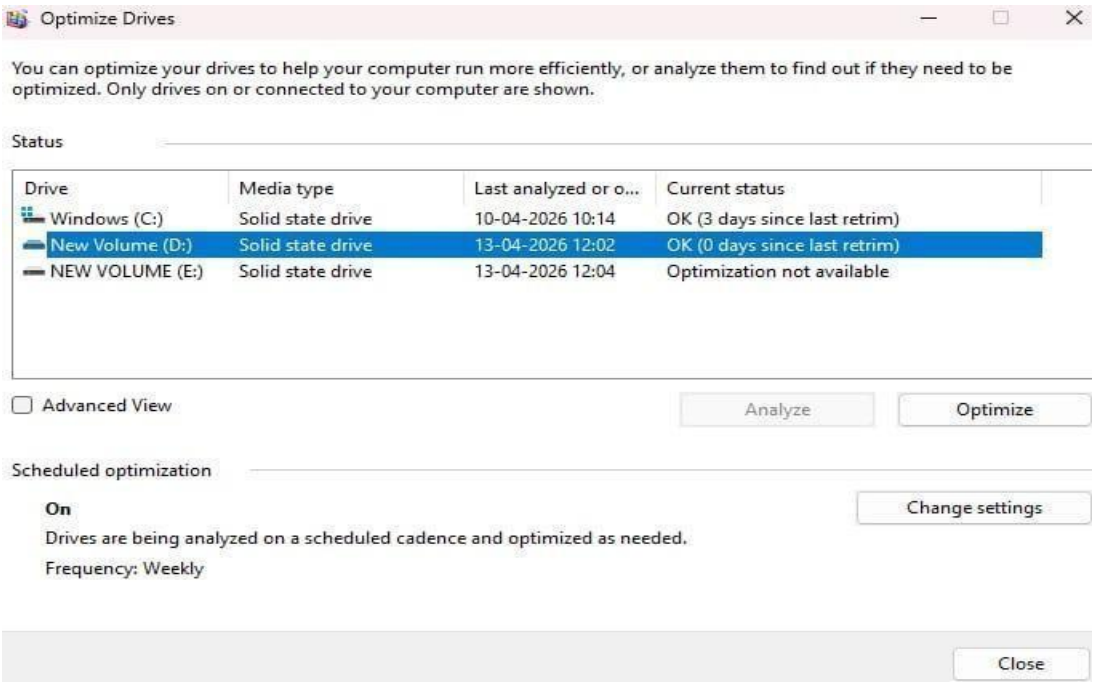


Fig 5.2.9: Disk Optimization Status Display

Fig 5.2.7 shows the disk optimization status of the storage volumes. The display confirms the health status and optimization condition of both FAT32 and NTFS partitions. This information helps evaluate the overall maintenance requirements of the file systems.

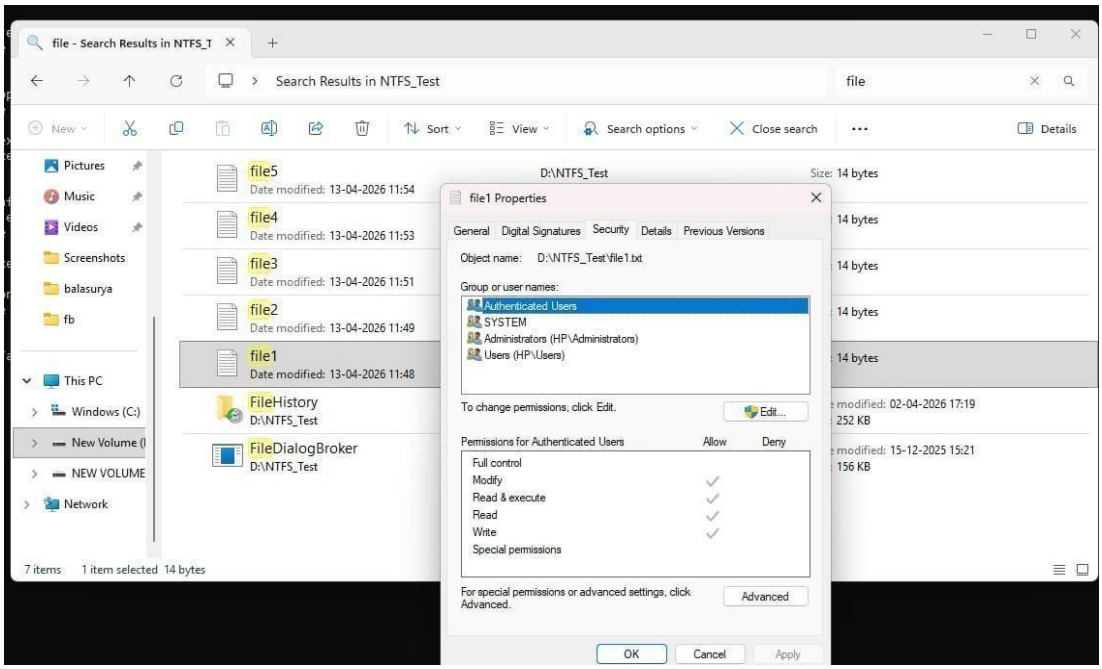


Fig 5.2.10: NTFS File Security Permission Display

Fig 5.2.10 illustrates the security permission settings available in the NTFS file system. Multiple user roles and permission levels are visible, demonstrating the advanced security features supported by NTFS. These features provide better protection and controlled access to stored data.

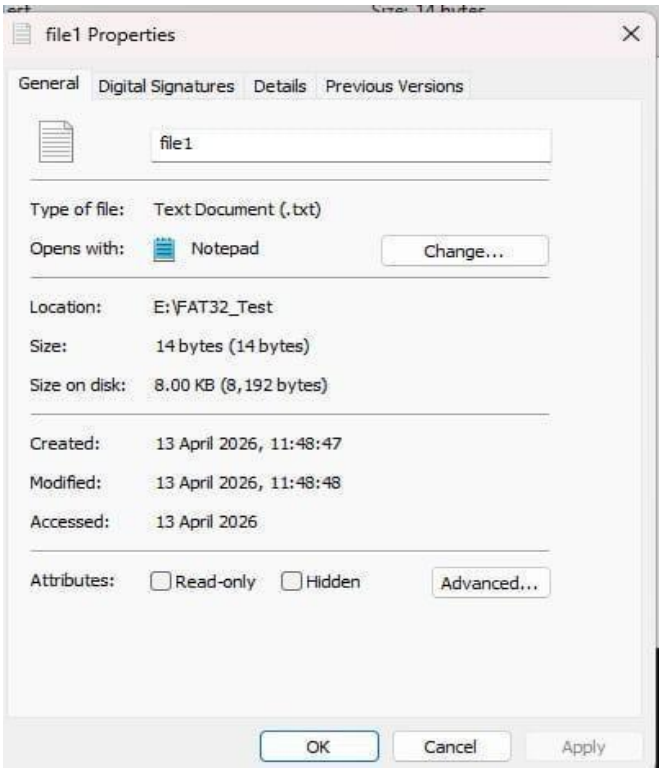


Fig 5.2.11: FAT32 File Properties Display

Fig 5.2.11 displays the file properties of a file stored in the FAT32 partition. The window shows file size and storage allocation details. Unlike NTFS, FAT32 does not support advanced security permissions, indicating its simpler design.

5.2 DISCUSSION

The results from our experimentation have made it clear that NTFS is better suited for file management and reliability in today's storage needs. With more advanced allocation strategies, there will be less fragmentation and better storage continuity. The result of which will mean increased speed in data access and better performance altogether.

In contrast, although simple, FAT32 does not have as many capabilities as NTFS in managing large quantities of data efficiently. Without features such as journaling, it makes it difficult for us to rely on FAT32 as our file system in critical cases. Yet it still has its purposes as far as removable media goes since compatibility takes precedence in those situations.

Through our methods of testing and analyses conducted throughout the project, we have established that file system choice is an important factor in assessing storage performance and reliability. It is important for us to know about the differences of FAT32 and NTFS in order to make a proper choice of file system according to our storage needs.

CHAPTER 6

CONCLUSION AND SCOPE OF FUTURE WORK

6.1 CONCLUSION

The project focused on analyzing and comparing the performance of FAT32 and NTFS file systems based on different file operations. The experiments were conducted by performing file creation, copying, deletion, and transfer operations using storage devices formatted with FAT32 and NTFS. The study helped in understanding the working principles and storage behavior of both file systems.

From the results obtained, it is observed that FAT32 provides faster performance for simple file operations and is suitable for small storage devices. However, FAT32 has certain limitations such as file size restrictions, increased fragmentation, and lack of advanced security features. These limitations reduce its performance when handling large files and repeated operations.

NTFS showed better performance and reliability. The use of the Master File Table (MFT) and journaling mechanism improved file management and reduced data loss risks. NTFS also supports large file sizes and

advanced security features, making it suitable for modern storage systems.

Overall, this project concludes that NTFS is more efficient and reliable for handling large-scale storage operations, while FAT32 remains useful for simple storage applications that require compatibility with multiple devices. The knowledge gained from this study helps in selecting the appropriate file system based on storage requirements and performance needs.

6.2 FUTURE SCOPE

The present project focused on analyzing the performance and storage behavior of FAT32 and NTFS file systems. In the future, the study can be extended to include additional file systems such as exFAT and APFS to their performance under different storage conditions. This will help provide a broader understanding of modern file system technologies.

Further improvements can be made by testing the file systems using different types of storage devices such as solid-state drives (SSD) and external hard disks. This will help analyze how different hardware environments affect file system performance and storage efficiency.

In future work, automated tools and advanced monitoring software can be used to measure performance parameters such as input/output speed, fragmentation level, and recovery time more accurately. These enhancements will help in developing more efficient storage management systems and improving overall file system performance.

REFERENCES

1 Raksha Rane and Asmit Singh,“Demystifying File Systems: A Comprehensive Exploration of Data Organization,”International Journal of Computer Science and Information Technology, 2024.

2 Li Qiyang, Zhang Quanxin, Tan Yu-an, Li Yuanzhang, and Zheng Jun,“Research on Allocator Strategy of FAT32 File System Based on Linux & Windows,”International Journal of Computer Applications, 2016.

[3] Microsoft Corporation,“NTFS Technical Reference Guide,”Microsoft DevelopDocumentation, 2019.

4 Andrew S. Tanenbaum and Herbert Bos,“Modern Operating Systems,”4th Edition, Pearson Education, 2015.

5 William Stallings,“Operating Systems: Internals and Design Principles,”9th Edition, Pearson Education, 2018.

6 Liu Naiqi, Wang Zhongshan, Hao Yujie, and Qin Ke,“Computer Forensics Research and Implementation Based on NTFS File System,”International Conference on Computer Science and Software Engineering, 2008.

7 W. Y. Lee,“Extraction of Creation-Time for Recovered Files on Windows FAT32 File System,” Applied Sciences, vol. 9, no. 24, pp. 1–17, 2019.

8 M. P. Mehta, “Formalising File systems in the ACL2 Theorem Prover: An Application to FAT32,” arXiv Computer Science Archive, pp. 1–20, 2018.

9 M. Karresand et al., “Disk Cluster Allocation Behavior in Windows NTFS,” Digital Investigation Journal, pp. S3–S11, 2019.

10 W. Lee and J. Lee, “File Allocation Characteristics in Windows FAT32 Systems,” International Journal of Computer Applications, 2018.

11 A. Sharma and R. Kumar,“Analysis of File System Allocation and Storage Management,”IEEE International Conference on Computing Technologies, 2017.

12 S. Patel and D. Shah,“Performance Evaluation of Modern File Systems in Storage Devices,”IEEE International Conference on Data Engineering, 2016.

13 H. Kim et al.,“File System Fragmentation and Storage Optimization Techniques,”IEEE Storage Systems Journal, 2018.

14 L. Chen et al.,“Efficient Disk Storage Management Using Advanced File Systems,”IEEE Transactions on Computers, 2020.

15 D. Wang and Y. Liu,“Storage Efficiency Analysis of FAT-Based File Systems,”IEEE International Conference on Big Data Storage Systems, 2021.

SDG GOALS ADDRESSED – MAPPING

The proposed project contributes to several United Nations Sustainable Development Goals (SDGs) by improving the efficiency, reliability, and management of digital storage systems. Efficient file system selection and storage management help optimize system performance, reduce storage waste, and support the development of modern digital infrastructure.

SDG 9 – Industry, Innovation and Infrastructure

This project directly supports SDG 9, which focuses on building reliable infrastructure and promoting innovation in technology. The study of FAT32 and NTFS file systems helps improve the efficiency of data storage and management in computer systems. By analyzing storage allocation strategies and performance behavior, the project contributes to the development of improved storage technologies used in industries such as cloud computing, data centers, and enterprise systems. Efficient file systems are an essential part of modern digital infrastructure, and understanding their performance helps promote technological innovation and system reliability.

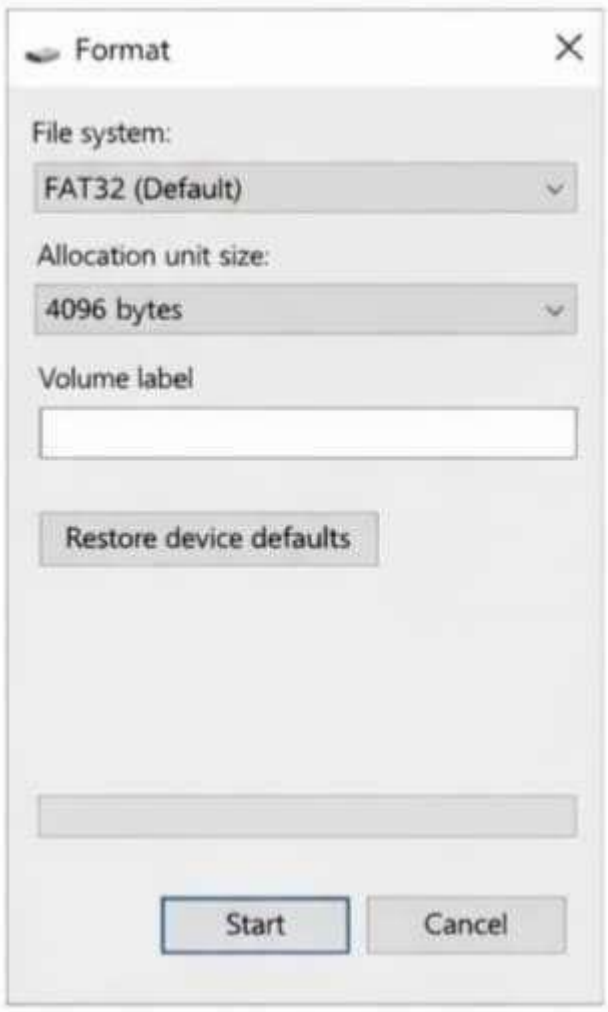
SDG 12 – Responsible Consumption and Production

The project contributes to SDG 12 by promoting efficient usage of digital storage resources. Through the analysis of fragmentation behavior and storage allocation methods, the project helps identify ways to reduce storage wastage and improve resource utilization. Efficient storage management reduces unnecessary data duplication and improves the lifespan of storage devices. By optimizing the usage of available storage space, this project supports responsible consumption of digital resources and promotes sustainable computing practices.

SDG 4 – Quality Education

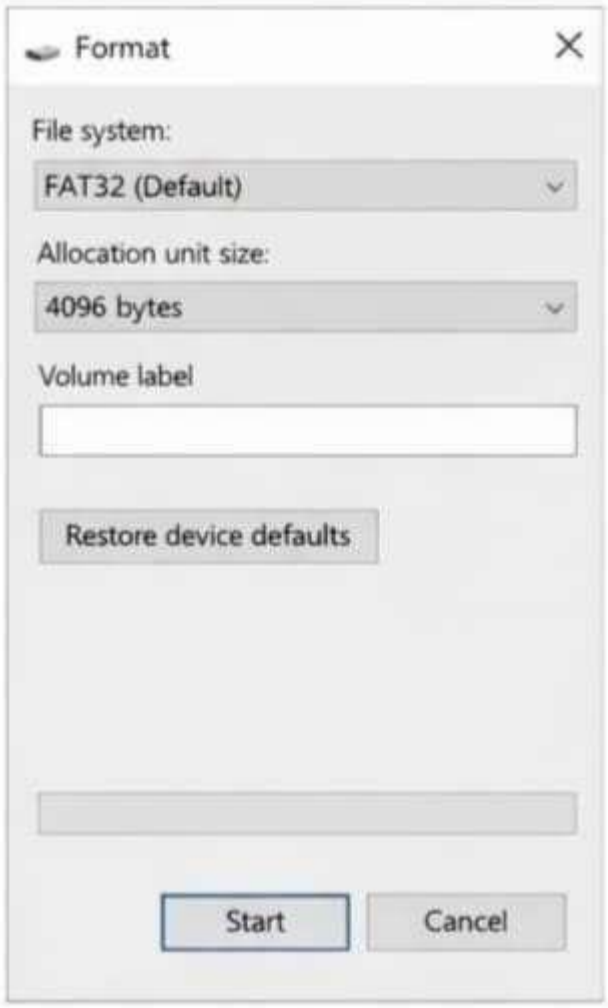
This project also supports SDG 4, which focuses on providing quality education and improving technical knowledge. The study of FAT32 and NTFS file systems enhances understanding of file system architecture, storage management, and performance analysis techniques. The knowledge gained through this project helps students develop problem-solving skills and practical understanding of operating system concepts. Such technical learning contributes to improving educational outcomes and prepares students for real-world technological applications.

ANNEXURE I



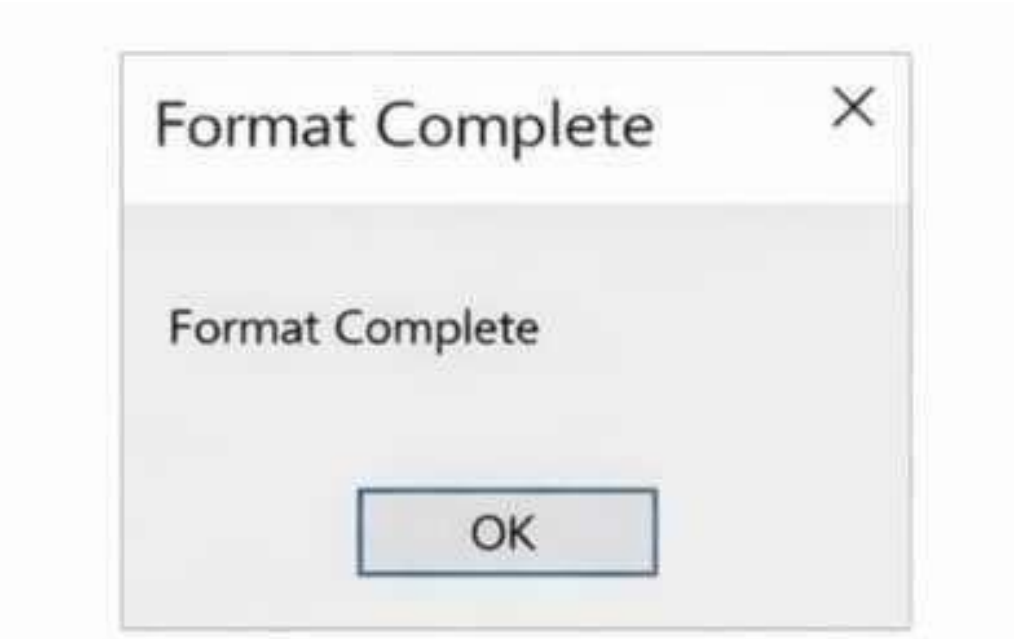
FigA1: Formatting Storage Device using FAT32

FigA1 illustrates the process of formatting the storage device using the FAT32 file system. In this step, the FAT32 option is selected from the file system dropdown menu in the format window. FAT32 is widely used for removable storage devices due to its compatibility with multiple operating systems. The allocation unit size is set to the default value, and the formatting process is initiated by selecting the start option. This step prepares the storage device to support file operations using the FAT32 file system.



FigA2: Formatting Storage Device using NTFS

FigA2 shows the formatting process of the storage device using the NTFS file system. NTFS is selected from the file system menu in the format window to enable advanced storage features. NTFS supports larger file sizes, improved storage efficiency, and enhanced security features to FAT32. After selecting the NTFS option, the formatting process is started, and the storage device is configured to support NTFS file system operations. This step ensures better reliability and efficient file management.



FigA3:Format Completion Message

FigA3 displays the format completion message that appears after the formatting process is successfully completed. This message confirms that the storage device has been formatted using the selected file system. After receiving the confirmation message, the storage device becomes ready for performing file creation, copying, deletion, and transfer operations. This step indicates that the device is successfully prepared for further experimental procedures in the project.

ANNEXURE II

BASE PAPER

Mobile Networks and Applications (2020) 25:248–258
https://doi.org/10.1007/s11036-019-01441-1



Disk Cluster Allocation Behavior in Windows and NTFS

Martin Karresand[1,2] · Stefan Axelsson[1,3] · Geir Olav Dyrkolbotn[1]

Published online: 19 December 2019
© The Author(s) 2019

Abstract
The allocation algorithm of a file system has a huge impact on almost all aspects of digital forensics, because it determines where data is placed on storage media. Yet there is only basic information available on the allocation algorithm of the currently most widely spread file system; NTFS. We have therefore studied the NTFS allocation algorithm and its behavior empirically. To do that we used two virtual machines running Windows 7 and 10 on NTFS formatted fixed size virtual hard disks, the first being 64 GiB and the latter 1 TiB in size. Files of different sizes were written to disk using two writing strategies and the $Bitmap files were manipulated to emulate file system fragmentation. Our results show that files written as one large block are allocated areas of decreasing size when the files are fragmented. The decrease in size is seen not only within files, but also between them. Hence a file having smaller fragments than another file is written after the file having larger fragments. We also found that a file written as a stream gets the opposite allocation behavior, i. e. its fragments are increasing in size as the file is written. The first allocated unit of a stream written file is always very small and hence easy to identify. The results of the experiment are of importance to the digital forensics field and will help improve the efficiency of for example file carving and timestamp verification.

Keywords Digital forensics · File carving · Allocation algorithm · NTFS

1 Introduction

File carving [17, 18] and timestamps are two key concepts in digital forensics. Unfortunately both concepts are laden with complex operations and a large amount of uncertainty regarding the correctness of the results. In file carving the digital forensic investigator has to find and categorize a large amount of (fragmented) data and then put the fragments back into the original file again without the help of a file system. The same applies to timestamps, where a timeline must be constructed, often only with data from different log files and the file system meta data at hand, two sources that are easy to manipulate. In both cases a digital forensic investigator would be helped by the extra information hidden in the storage structure of data, its allocation layout.

The file carving process and the extraction of timestamps are both dependent on the behavior of the combination of file system and operating system (OS), which govern the placement of data on hard disks through an allocation algorithm. Knowledge of the layout pattern of data on disk is crucial to the forensic investigator when doing file carving. However, the actual behavior of the allocation algorithm is not known. For example the general assumption used as a basis for different forensic file carving tools is that the data to be carved is stored contiguously and is only mildly fragmented.

To improve the timestamp information used in investigations several propositions have been made to also use the behavior of the allocation algorithm as a source [25, 26]. However, we have not found any previous work that study the exact behavior of the allocation algorithm of for example New Technology File System (NTFS). There is a small number of main allocation algorithm concepts used in all modern OSs, but the exact behavior of the different

The research leading to these results has received funding from the Research Council of Norway programme IKTPLUSS, under the research and development project Ars Forensica grant agreement 248094/O70.

✉ Martin Karresand
martin@filecarving.net

1 Department of Information Security and Communication Technology, Norwegian University of Science and Technology, Gjovik, Norway

2 Division of Command, Control, Communications, Computers, Intelligence, Surveillance and Reconnaissance, Swedish Defence Research Agency, Linkoping, Sweden

3 School of Information Technology, Halmstad University, Halmstad, Sweden

implementations of the algorithms are not known, at least not for the closed source OS variants. Windows using NTFS is for example said to use a best fit allocation strategy, but that information is getting dated and is also based on the Linux implementation of the NTFS driver [1]. We therefore have studied the allocation behavior of two modern versions of Windows (7 and 10) in combination with NTFS to empirically reverse engineer the allocation algorithm(s) used. The data source is based on writing files of different sizes to disk using different writing strategies (writing the file as one large block at once or as a continuous stream of data). The experiment was done using two virtual machines having fixed size virtual hard disks of different sizes, one 64 GiB and one 1 TiB.

The experiment is part of the future work presented in two earlier articles [5, 11] where we develop a framework to create maps of user data placement on hard disks. The maps give the probability of finding unique user data at different Logical Block Addressing (LBA) positions in Windows NTFS partitions and can for example be used for triage, planning of hard disk investigations and to enable different areas to be prioritized in file carving processes. Knowledge of the actual behavior of the allocation algorithm of different file systems will enhance the precision of the maps.

Our work is based on an empirical evaluation of the behavior of the allocation algorithm of Windows NTFS, using the hypothesis that a best fit allocation strategy is used. Hence we do not reverse engineer any code or expose any secret information. Likewise the reported results are not detailed enough to enable someone to recreate the Microsoft Windows NTFS allocation algorithm or driver. However, the presented results are of great value to the digital forensics community and we therefore make them public as a help in the global fight against digital crime.

The rest of this paper is organized as follows: The remaining parts of Section 1 presents background on file allocation algorithms and related work. In Section 2 we describe how the experiment was implemented. Section 3 presents the results of the experiment and in Section 4 we discuss the effects and implications of our result to the research field of file carving and other relevant areas within and related to digital forensics. Section 5 concludes the work and presents ideas of future work to be done.

1.1 Background

The theory of file system construction is for example described by Silberschatz et. al [20], Stallings [21] and Tanenbaum and Bos [23]. A file system keeps track of data stored on secondary storage and is organized in different ways. However, all implementations share some common properties: the addressing of the physical storage is abstracted by the file system into logical addresses and the logical storage position of written data is determined by an allocation algorithm.

During the history of computer systems different methods of allocating disk space have been in use. The main methods are contiguous, linked and indexed allocation [20, 21, 23]. The indexed allocation strategy, where the addresses of the data blocks are held in an index separated from a file's data, is currently the most popular. The indexed allocation strategy does not suffer from external fragmentation (unallocated holes being to small to be filled with new data), but heavily used storage media can still lead to fragmentation of files and require regular defragmentation. There is also a risk of disk space being wasted when using indexed allocation, especially for small files requiring a full index meta data block to hold just a few index posts.

There are also a number of algorithms used for handling the free space that is to be populated by new files. Silberschatz et. al [20] presents three algorithms; *first fit*, *best fit* and *worst fit*, Stallings [21] mentions *nearest fit* and Tanenbaum and Bos [23] add *next fit* and *quick fit*. Together they give the following free space allocation algorithms:

First fit starts the search for free space at the beginning of the file system and has the requirement of the space being large enough to hold the entire file. If there is no space large enough to hold the entire file, a fragment is written at the found position and the search continues.

Next fit (Stallings [21, p. 544] calls this *nearest fit*) uses the same principle as *first fit* of allocating the next free space being large enough to hold the entire file, but the search is done from the current position in the file system. If there is no single free space to hold the entire file the file is fragmented and the available free spaces are used to hold the file.

Best fit uses the free space best fitting the new file, i. e. giving the smallest remaining free space. This requires all the available free spaces to be scanned before the best fit can be chosen.

Worst fit uses the free space having the worst fit to the new file. This is the opposite to best fit, i. e. giving the largest remaining free space. As for the best fit algorithm the available free spaces of the entire file system have to be scanned before the worst fit can be chosen.

Quick fit uses several lists of different sizes of free block areas. In this way a fitting area can be found very quickly, but the algorithm suffers from a complex process during deallocation when freed up areas might have to be merged. If this is not done the storage will soon fragment into a large amount of free areas too small to be usable.

These free space allocation algorithms are not specific to storage of data on disk, they are also used in for example

memory allocation in Random Access Memory (RAM) [20].

Based on the information given by [3] and [15] Microsoft Windows' NTFS is using an index allocation strategy. The problem of space being wasted when using index allocation is solved in NTFS by storing the data of smaller files (up to approximately 700 bytes)[1] in the Master File Table (MFT) meta data records themselves [15].

According to Carrier [1] the best fit free space algorithm is used by Windows XP on NTFS formatted hard disks. Since the book was written in 2005 it does not cover the allocation algorithms used by Windows 7 and newer. Our previous experiments [5, 11] indicate that the actual behavior of the allocation algorithm in NTFS is not strictly best fit. The results from experiments indicate that groups of free clusters are allocated in descending order of size, which is best fit, but not always. The deviating behavior is mentioned in a Superuser Q&A [22].

When formatting an NTFS partition 12.5% of the space is by default reserved for the MFT [15]. The MFT records are 1 KiB in size and usually the size of the smallest allocatable unit (called cluster) in NTFS is 4 KiB [15]. The allocation status of every cluster in the file system is stored in the $Bitmap file, which is record number 6 in the MFT. Each bit in the $Bitmap file represents one file system cluster in ascending LBA order. If a cluster is allocated the corresponding bit in the $Bitmap file is set to 1, hence 0 represents an unallocated cluster.

Files can be written to disk in two ways; either as a stream, or the entire file as a block. In the first case the OS does not know the final size of the file and therefore cannot optimize the allocation accordingly. This often leads to file fragmentation, but the behavior is partly mitigated by the internal buffering of the OS. When the entire file is written in one piece the OS knows its size in advance and can utilize the standard allocation algorithm, which optimizes the storage. This behavior is probably more common when dealing with smaller files that easily can be held in RAM, than for large files. The specific write behavior is software dependent and might incorporate temporary writing of files to protect the data in case of a power loss or hardware failure.

1.2 Related work

We have not found any related work directly addressing the detailed behavior of the allocation algorithm in Microsoft Windows 7 and 10 running in NTFS formatted partitions. There are however some work done on the connection between timestamps, cluster allocation and file creation order.

Willassen studies the formal foundation of timestamps in his PhD thesis [26]. There he also formulates the criteria needed to use the allocation strategy of a file system for checking timestamps. He also briefly discusses the allocation strategy of NTFS and states that it is best fit in Windows XP and that a first fit allocation strategy is used within the MFT. The PhD thesis is partially based on an earlier article by Willassen [25] describing a method to use a first fit allocation strategy to detect antedating of files.

Tse [24] intends to use the allocation pattern of files to estimate the causal ordering of events, but concludes that using the actual allocation patter is complex. Tse instead defines two metrics approximating the allocation pattern causality and then tests the metrics against standard timestamps.

Minnaard [16] studies the inner workings of the Linux implementation of the File Allocation Table (FAT32) file system in an article from 2014 by looking at the source code and then uses the result to find the causal order of files from a TomTom Go. The article mentions that the Windows 7 implementation of FAT32 is more complex than the Linux variant.

2 Experimental setup

The experiment is designed to test whether Windows in combination with NTFS is using the best fit allocation strategy as indicated in the literature [1, 26]. To enable this the behavior of the allocation algorithm has to be studied in different situations regarding disk utilization, file system fragmentation, file size and partition size. We therefore manipulated the $Bitmap file of the file systems of the virtual machines used to emulate different states of file system fragmentation.

2.1 Virtual hardware

The experiments were performed on two virtual machines using VirtualBox running Windows 7 and Windows 10. The Windows 7 machine had a fixed size 64 GiB hard disk and the Windows 10 machine a 1 TiB ditto to cover both older and newer Windows OSs, as well as small and large hard disks. Each virtual machine was given a folder shared with the host. To enable us to use standard digital forensic tools (the Sleuth Kit [2]) on the virtual hard disks and their partitions they were loop mounted with read/write access rights.

The disk of the Windows 7 machine was already used since it was taken from one of our earlier experiments [5],

[1] The maximum size of an internal $Data attribute varies depending on the size of other attributes stored in the MFT record. Most sources give a maximum internal $Data attribute size of 600 to 700 bytes. Microsoft reports a 900 byte limit [15].

where 10000 file operations were performed in a pseudo-random pattern with bias towards file writing. This had given a heavily fragmented file system corresponding to an old and well used (home) computer. The fragmentation status of the file system was checked before the start of the experiment and one remaining large area of free clusters was found at the end of the partition. To even out the fragmentation pattern we decided to split that area into five smaller areas of approximately 262000 clusters each, corresponding to approximately 1 GiB per area.

The Windows 10 machine was freshly installed and the file system therefore only contained files from the installation, without any significant fragmentation. It represented an office computer using mainly network based storage, without synchronization between local and network storage.

2.2 Process description

During the experiment each virtual machine was repeatedly power cycled to ensure that all file operations were flushed to hard disk (both the virtual and the real on the host). The experiment started by copying the $Bitmap file of the powered down virtual machine to the host. Then the following steps were iterated over for each file write operation:

1. The virtual machine was powered on
2. A file signaling the power on sequence had finished was written to the shared folder
3. The file write operation was executed
4. A file signaling the file operation had finished was written to the shared folder
5. The virtual machine was powered down
6. The power down status was checked using `vboxmanage`
7. The allocation information of the newly written file was extracted using the `istat` [2] tool
8. The $Bitmap file of the virtual machine was copied to the host using `icat` [2] tool

To allow the virtual machine, as well as the host, to properly write all files to disk up to 10 second long delays were introduced between the steps 1 to 3 of each iteration. We also used files written to a shared folder to signal when a step was finished.

To cover for both writing a file as one block and writing it as a stream a Python 2.7 script was used to either write a file to an array in RAM and then to disk (block writing) or using Python's own file write operation directly writing one 512 B sector at a time (stream writing). Both types of write operations were buffered by the OS, but in the block writing case the OS got information on the file size before the file was written to disk, which it did not when the file was written as a stream. Each 512 byte block of the files were uniquely marked with a consecutive number together with a file identifier. The marking was used as a backup procedure in case we had to read the raw data from the virtual hard disks due to errors in the process. The marking was also used to verify that the `istat` tool reported the allocation pattern in the same order as the clusters were written to, which it did.

We used files of different sizes to determine if the allocation algorithm behaved differently depending on the size of the written file. In the first round of the experiment we used 4, 128, 511, 512, 513 and 1024 MiB files. The 511 and 513 MiB files originated from the first version of the Python script, where the type of writing was depending on the size of the file. If the file size was $\leq$ 512 MiB the file was written as one block and if it was larger it was written as a stream. The range of file sizes were later expanded to also include 12, 96, 384, 768 and 1536 MiB files to get a more even coverage of small and large files and the Python script was updated to enable different writing strategies regardless of file size.

2.3 Bitmap manipulation

To cover for possibly different allocation behavior depending on the available amount of storage of the file system we manipulated the $Bitmap file of the Windows NTFS virtual hard disks. The $Bitmap file of the Windows 7 machine was manipulated once and the Windows 10 $Bitmap file three times.

The Windows 7 virtual machine's main partition was heavily fragmented from the beginning, but still contained a contiguous area of approximately 5.3 GiB. This area was divided into five smaller areas, which can be seen in Table 1, all other free areas where kept in their original state. The modified layout is called *BM 7:1* throughout the text. The unmodified layout was never used due to its already heavily fragmented file system.

Directly after installation of the Windows 10 virtual machine its 1 TiB main partition contained two large unallocated areas of approximately 497 and 511 GiB respectively at the

Table 1 The unallocated areas in the original 5.3 GiB space after the *BM 7:1* manipulation

Start position	Size [cluster]
15372418	262000
15634546	262143
15896818	262144
16159090	262145
16421363	262160

Table 2 The modified areas of *BM 10:2*

Spaces	Function	Tot. [cluster]
511	int$(120000/x + 27; x = [512 : -1 : 2])$	711541
23	120000	2760000
512	$7x + 13; x = [0 : 511]$	922368
16	7999	127984
1	17	17
29	int$(1800000/x + 17; x = [1 : 29])$	7131462

end of the partition. This original, unmodified, layout is called *BM 10:0* throughout the text.

After the *BM 10:0* file writing operations were executed we manipulated the $Bitmap file to fragment the allocation layout. The smaller of the two large free areas was divided into 501 equally sized unallocated areas of 120000 clusters (468.75 MiB) each and the larger unallocated area was divided into 1026 unallocated areas of increasing size, from 120 clusters (480 KiB) to 123120 clusters (approximately 481 MiB) in steps of 120. The two areas together contained 123342120 (approximately 471 GiB) free clusters after the modification. All other unallocated areas on the partition were unmodified. We refer to this manipulation setting as *BM 10:1*

After testing the *BM 10:1* allocation manipulation we created a new bitmap (referred to as *BM 10:2*) file for the Windows 10 virtual hard disk where we decreased the available space even more by first restoring the virtual machine to its original state and then creating the free areas shown in Table 2 from the two large unallocated spaces.

The *BM 10:2* manipulation was meant to test the block writing allocation behavior by forcing the algorithm to chose between a few very large areas and many small. The seemingly odd values used in Table 2 were chosen to avoid creating free areas of exact multiples of 2. The total amount of free space in the manipulated area was 11715760 clusters (44.7 GiB) after the modification.

However, the *BM 10:2* $Bitmap manipulation was not strict enough to generate any significant fragmentation during the block writing. We therefore manually decreased any remaining free areas larger than 120000 consecutive clusters with a factor 10. This left a total of 3361315 clusters (12.8 GiB) of free space in the manipulated area. This last manipulation is referred to as *BM 10:3*.

3 Result

The overall result of the experiment shows that Windows 7 and 10 using NTFS formatted partitions are both using a best fit allocation algorithm. This is however not always true, as will be shown in the following sections. In the text we will refer to *lower rest of free clusters* as a term for allocation patterns where the pattern gives a lower number of remaining free clusters compared to a strict best fit pattern. Please observe that the term is only defining a local minimum, i. e. we have only looked at one alternative pattern.

3.1 Block writing

The result of the block writing operations are found to be following the best fit allocation strategy in most of the cases in Windows 10, but not for Windows 7. We also present an interesting pattern common to both versions of Windows regarding the sizes of fragments.

3.1.1 Windows 7

The result of the block writing file operations on the 64 GiB hard disk of the Windows 7 virtual machine using *BM 7:1* can be seen in Fig. 1. The file system was already partially fragmented due to 10000 random file operations executed in an earlier experiment and we fragmented the remaining large free area into five smaller ones to put an even higher burden on the allocation algorithm.

All but three block writing file operations result in between 2 and 9 fragments, which are all allocated in descending order of size within each file operation (apart for file operations 5 to 7 (fragments 14 to 16 in Fig. 1), which are small and unfragmented). None of the fragmented file operations are best fit allocations and neither are they optimized from the point of lower rest of free clusters. The last fragment in every file operation is smaller than the previous fragments in the operation, which is represented by the dips in the curve in Fig. 1. What also can be noticed

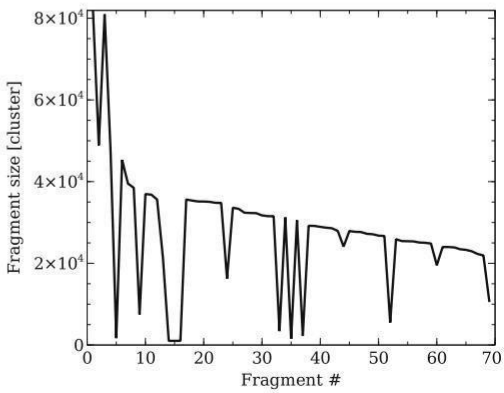


Fig. 1 The decrease in fragment size for the Windows 7 block writing experiment using the *BM 7:1* layout

is the decreasing size of the fragments even between file operations, where the size of the second last fragment in the previous file operation is larger than the starting fragment size in the following file operation. Also the free areas chosen by the algorithm are often slightly larger ($<$ 10 clusters) than the fragment and therefore free space might be wasted.

The continuously decreasing fragment sizes seen in Fig. 1 might seem to indicate a typical best fit behavior, but there were larger free areas that would have been better to use from a best fit point of view. Instead the algorithm seems to balance the sizes of the fragments to be as similar as possible, apart from the last fragment. We did not see any pattern regarding the position of the chosen fragments.

3.1.2 Windows 10

The original installation (*BM 10:0*) of Windows 10 on a 1 TiB virtual hard disk contains two large areas of free clusters at the end separated by a 512 MiB large area holding a number of files related to the System Volume Information directory. The two areas are the largest free areas on disk and over 500 times larger than the third largest area. The 11 largest free areas on the partition can be seen in Table 3.

All file operations using *BM 10:0* allocated files of one block each, which can be seen in Table 4. The first and second write operations are best fit allocations, as can be seen when comparing their sizes and positions to the available free areas in Table 3. The rest of the file write operations all allocate consecutive areas in the second largest free area, which is not a strict best fit behavior. Even the three 4 MiB file write operations (operation 3 to 5 in Table 4) are allocated in the same area and not in any of the better fitting available areas. Although there are system related and stream writing file operations executed between operations 3 to 5 there are several free areas that would fit the file sizes of these operations better than the actual allocation do.

After the file write operations in *BM 10:0* 19 block write operations are performed using the *BM 10:1* allocation layout. The larger write operations are split into $n \geq 2$ fragments, where the $n - 1$ first fragments all have approximately the same size within each operation. The largest available free area is 123120 clusters in size and as can be seen in Table 5 that area is the first to be allocated in this part of the experiment.

All positions in Table 5 ending in "4544" are free areas created by us. The larger areas belonging to the "4544" series are also allocated in descending order, hence they are allocated in best fit order. Also the smaller, remaining parts, of the file write operations belonging to the "4544" series are allocated in best fit order, although some of them do not fully occupy the original free area. The unfragmented file operation 6 is also a best fit allocation due to a number of previous system file deallocations that left a free area of suitable size.

The part of the experiment using the *BM 10:2* allocation layout results in single block allocations and all of them are made in order of best fit. Thus even eight 768 MiB files all result in single block allocations. Worth noticing is the fact that all but one allocation are made within the modified area, i. e. not in any of the original free areas. The best fit allocation approach is proven by the fact that an area outside of the modified area is used in a sequence of equally sized files. That area has been available since the installation of the OS and has a size that fits in between the sizes of the modified free areas.

The execution of the file operations using the *BM 10:3* allocation layout gives files fragmented into 3, 4 and 5 fragments, as can be seen in Table 7. The allocation strategy is no longer strictly best fit. Fragment 2 in file operation 1 is not best fit, because the free area at position 264466712

Table 3 The 11 largest free areas in the original installation of Windows 10 on a 1 TiB hard disk (*BM 10:0*)

Position [cluster]	Free size [cluster]
150728	869
135305	1259
2009009	1679
2005999	3009
2010944	3742
2570502	4774
56466	14272
809984	27680
2775255	239688
3021071	131132401
134284544	134022399

Table 4 The 8 block write operations performed using the original allocation layout *BM 10:0*. The operations are presented in order of execution

File op.	Position [cluster]	Size [cluster]
1	2775255	131072
2	3148190	131072
3	3280413	1024
4	3283997	1024
5	3282700	1024
6	3291406	262144
7	3815408	32768
8	3882725	32768

Table 5 The 19 block write operations performed using the *BM 10:1* allocation layout. The operations are presented in order of execution

Op.	Frag.	Pos. [cluster]	Size [cluster]
1	0	267534544	123120
1	1	142864544	7952
2	0	267404544	123000
2	1	142994544	8072
3	0	267274544	122880
3	1	267144544	122760
3	2	152094544	16504
4	0	267014544	122640
4	1	266884544	122520
4	2	152614544	16984
5	0	137534544	3072
6	0	3553550	3072
7	0	160804544	24576
8	0	137664544	3072
9	0	160934544	24576
10	0	161064544	24576
11	0	161194544	24576
12	0	240754544	98304
13	0	240884544	98304
14	0	241014544	98304
15	0	241144544	98304
16	0	266754544	122400
16	1	266624544	122280
16	2	266494544	122160
16	3	162754544	26376
17	0	266364544	122040
17	1	266234544	121920
17	2	266104544	121800
17	3	163924544	27456
18	0	265974544	121680
18	1	265844544	121560
18	2	265714544	121440
18	3	165094544	28536
19	0	265584544	121320
19	1	265454544	121200
19	2	265324544	121080
19	3	166264544	29616

would have been better to use, as can be seen in Table 6 showing the 21 largest free areas available to file allocation 1. The second file allocation in Table 7 is strictly best fit, fragments 0 to 2 belong to the largest available free areas in that round and fragment 3 to the free area best fitting the remainder. The last file allocation, number 3 in Table 7 is not best fit at a first glance. There is a fragment of 56801 clusters that would have been better to use. However that free area would then have been combined with an area leaving 1589 free clusters. The current allocation only leaves 205 clusters unallocated and therefore has a lower rest of free clusters than a potential strict best fit allocation. If this is the actual behavior of the allocation algorithm is however not possible to deduce from only one file operation.

Table 6 The 21 largest free areas in the (*BM 10:3*) allocation layout. These are also all free areas equal to or bigger than 18017 clusters in the partition

Position [cluster]	Free size [cluster]
265866802	18017
124324183	24027
265638022	28409
124564183	30027
124804183	40027
263669912	40407
264466712	56801
265380686	60550
267499096	62144
267646484	69247
267574290	72017
267338630	81835
267252722	85731
262618164	85961
267162528	90017
267067598	94753
266967404	100017
265080492	103409
266861328	105899
125536472	107711
266748634	112517

Table 7 The 3 block write operations performed using the *BM 10:3* allocation layout. The operations are presented in order of execution

Op.	Frag.	Pos. [cluster]	Size [cluster]
1	0	266748636	112512
1	1	125536472	107708
1	2	265380688	41924
2	0	267252724	85728
2	1	267338632	81832
2	2	267574292	72012
2	3	124324184	22572
3	0	267646484	69244
3	1	267499096	62140
3	2	267438388	60531
3	3	263669912	40407
3	4	124564184	29822

3.1.3 General observations

We have not been able to see any patterns in the allocation algorithm's behavior regarding how the allocated positions are chosen. Likewise we have not been able to find an algorithm for how the fragment sizes are chosen. In most cases the fragments are somewhat smaller (a few clusters) than the free area, leaving small free areas before and/or after the fragment. Hence the the positioning and size of the fragments within the free areas are probably governed by some rules, but we have to few data to fully deduce them.

All fragmented block writing operations have one feature in common, which is the globally decreasing fragment size. The second last fragment in a file operation is always larger than the first fragment in the following file operation. The decreasing size feature is also valid within each file operation. The feature can be seen in Fig. 1 for the Windows 7 virtual machine and in Tables 4, 5 and 7 for the Windows 10 virtual machine. This pattern is valid even if there are system file or stream writing operations interleaved with the block writing operations.

3.2 Stream writing

The result of the stream writing file operations consists of 15 operations done in Windows 7 and 23 operations in Windows 10. Since the files are written as a stream the OS cannot implement a proper best fit allocation strategy and we therefore do not check for it.

3.2.1 Windows 7

There is a weak correlation between file size, the file writing order and number of fragments in the results, which can be seen in Table 8. The correlation is however very weak and seems to be increasing for small files and decreasing for larger files.

The allocation patterns of the different file operations always begin with very small fragment sizes, often there is a single cluster allocated first. The size of the fragments increases as more data is written to disk. In some cases the fragment size is doubled in each of the first 5 consecutive allocations. This pattern diminishes as even more data is written, but the size of the fragments is constantly increasing, occasionally with a large deviation either up or down. The deviations are sometimes as frequent as every second allocation. In some cases the fragment size is suddenly increased with one or two orders of magnitude, a few times even more.

3.2.2 Windows 10

As for the Windows 7 case we show the number of fragments per file and their sizes in clusters (see Table 9).

Table 8 The number of fragments per file and their sizes in clusters for the Windows 7 stream writing experiment using *BM 7:1*. There is a weak correlation between the number of fragments, the writing order and the file size, which is most clearly seen for the largest files. The result is presented in order of size and time of writing

No. of frag.	File size [cluster]
26	1024
35	1024
40	1024
70	32768
44	32768
1131	131328
468	131328
195	131328
242	131328
339	262144
226	262144
1657	393216
340	393216
328	393216
206	393216

Windows 10 does not show any correlation between file size, writing order and number of fragments.

The stream writing file allocation for Windows 10 is smoother than for Windows 7, with fewer and lower deviations. The general increase of the fragment's size seen in Windows 7, as well as the small fragments at the beginning of the file operations, are present also in the Windows 10 results. The first few file fragments often reach sizes of approximately 40 clusters in two or three steps, which is faster than in Windows 7. The first stream writing file operations using the *BM 10:0* allocation layout ends with one very large allocation, especially when the files are bigger. In Table 9 this is manifested in the low number of fragments connected to some of the files containing 131072 and 262144 clusters.

3.2.3 General observations

Both Windows 7 and 10 show a general increase, although not without deviations, regarding the size of the allocated fragments. The allocation always starts with a small fragment, the maximum size of the first fragment for both versions of Windows is 5. The distribution of starting fragment sizes can be seen in Table 10.

All (100%) of the Windows 7 starting fragments are of size 1 and for Windows 10 the amount of size 1 starting fragments is 52%.

Table 9 The number of fragments per file and their sizes in clusters for the Windows 10 stream writing experiment using *BM 10:0*. No real correlation between the number of fragments, the writing order and the file size can be seen. The result is presented in order of size and time of writing

No. of frag.	File size [cluster]
7	1024
17	1024
27	1024
13	1024
48	1536
70	12288
18	32768
20	32768
174	49152
21	131072
39	131072
64	131072
43	131072
326	196608
165	196608
417	262144
1024	262144
103	262144
47	262144
23	262144
53	262144
48	262144
152	393216

4 Discussion

The main contribution of the experiment on the behavior of the NTFS allocation algorithm in Windows 7 and 10 is the result showing a globally decreasing fragment size of block writing file operations. However there are a few constraints that first must be fulfilled. First of all the file system must be fragmented, without any large areas of free, unallocated clusters and the files to be written have to be larger than the largest available free area to force the allocation algorithm to fragment them. Consequently there should not be any deallocations of areas larger than the next file to be written. If a new larger free area becomes available the decreasing trend will probably be restarted from there. Apparently stream writing interleaved with block writing is not affecting the decreasing size allocation pattern, but such files are not included in the decreasing pattern. Unfragmented files will also be excluded from the pattern.

Table 10 The accumulated amount of first fragment sizes in both Windows 7 and 10 using stream writing. All Windows 7 starting fragments are single clusters

Frag. size [cluster]	Amount [%]
1	71
2	16
3	8
4	0
5	5

The decreasing fragment size behavior can for example be used in digital forensic investigations to get a relative timestamp or the sequence of writing of a collection of block written files. By comparing the sizes of the first and second last allocated fragments of files their timestamps can be verified, or their internal age relative each other be decided.

It is also possible to separate a block written file from a stream ditto. If that knowledge is combined with knowledge on what type of writing strategy different software packages in a Windows computer use the probable source of a file can be decided. This is for example useful in triage situations where it will be enough to scan the NTFS meta data (the MFT records) to determine the main source of a file.

Also the stream writing behavior of the allocation algorithm can be of use for a digital forensic examiner. Our result shows that the size of the first fragment of a stream written file is 1 cluster in 71% of the cases (100% in the 64 GiB virtual hard disk running Windows 7) and none of the stream written files started with a fragment larger than 5 clusters. That corresponds to files between 4 and 20 KiB in size. We have earlier found the average amount of files in a standard office computer to be approximately 350000 by counting the files in 25 (office) computers running mostly Windows 7 [5, 11]. If we combine that with the official disk space requirement of 20 GiB for a standard Windows 7 to 10 installation [12–14] we get an average file size of 60 KiB. This is a theoretical lower bound, because we did not take the size of the user files included in the computer average of 350000 files into account. Hence any allocated area smaller than 60 KiB belongs to the first part of a stream written file with a high probability. This knowledge can for example be used in file carving processes to quickly find the data type of files directly from the data, without the need to trust the file name, by searching for small areas with similar data content and then extracting any magical bytes from them.

Since we have reverse engineered the allocation algorithm without access to any documentation we have no written proofs of the reasons behind the behavior we have found. We therefore can only hypothesize, but a valid reason for the block writing behavior of creating similarly sized fragments is to create an even distribution of the data over several cylinders or flash memory capsules for faster access and wear levelling.

The reason behind the stream writing behavior of starting small and increasing the fragment sizes might be two-fold. By starting from very small fragment sizes any free clusters left over when doing best fit allocation can be taken care of. The other reason is the fact that without knowledge on the actual size of the file to be written the algorithm has to estimate the final size of the file. By increasing the size of the fragments as more data is written we get a acceptable compromise between speed (less fragmentation) and a good utilization factor (small areas are not wasted). Hence the average fragment size will increase as the file size increases.

The file system allocation algorithm of Windows 7 and 10 when using NTFS should be best fit [1], but our experiment shows that it is not completely true. The block writing allocation is deviating from a strict best fit behavior in a few cases. Possible reasons for any differences in behavior are typically OS version, partition size, degree of file system utilization, file size and type of file writing behavior.

Regarding the influence of the OS we have not noticed any differences in allocation behavior. The result might have been influence by the fact that we only used two versions of Windows, but by using both the first and the latest versions of the latest generation of Windows we cover for any differences introduced in Windows 8 and 8.1.

Any differences in allocation strategy depending on the partition size is covered by the use of both a 64 GiB and a 1 TiB virtual hard disk. During the experiments we noticed small differences in behavior between the small and the large hard disks, especially in the stream writing case where the deviations from the increasing fragment size were smaller for the 1 TiB hard disk than for the 64 GiB hard disk. The most probable reason for this behavior is the larger amount of varied sizes of free areas to choose from in the larger disk.

We found that the allocation algorithm does not use a best fit strategy when allocating block written files in Windows 10 when using the unmodified *BM 10:0* allocation layout. Instead it allocates chunks of the large area of free clusters at the end of the file system, which seems reasonable from a wear levelling point of view. We did not observe the same behavior in the Windows 7 case, but there we had gotten rid of all large unallocated areas before the experiment started.

The allocation behavior during the 4 MiB file operations in Windows 10 (using the *BM 10:0* layout) where the algorithm allocates parts of one of the large free areas at the end of the partition cannot be explained by the fact that the block writing operations and the stream operations where interleaved during the experiment. Nor can it be explained by any system files being written to the free areas that would have been appropriate to use. After file operation 5 in Table 4 there still are several free areas of a few thousand clusters left that could have been used instead. The same applies to the file write operation 5 shown in Table 5. This behavior more resembles a worst fit allocation strategy than a best fit ditto.

The knowledge on the allocation behavior of Windows and NTFS gained through the experiments presented in this paper will benefit our previous work within the file carving area [4, 6–10]. There we experimented with different algorithms to detect the file type of data fragments using only the information held in the fragments themselves. Using for example the fact that the allocation algorithm has different behavior for block writing and stream writing can help identify and separate data types that are written in different ways. Also the fact that the free areas often are not fully utilized can help improve data type separation in heavily fragmented cases by explaining very small areas with different properties intertwined between larger areas of data with equal properties. Especially since it is well known that many file types contain areas of different types of data [4, 19].

5 Conclusion and future work

By writing files ranging in size between 4 MiB to 1,5 GiB to a Windows 7 virtual machine having a 64 GiB hard drive, as well as a Windows 10 virtual machine having a 1 TiB hard drive, we have found that the Windows NTFS allocation algorithm is more complex than the best fit strategy described in the literature [1]. In most of the file operations executed during our experiment the algorithm behaved as a strict best fit type, but when having access to a very large area of free clusters it started to allocate parts of that area instead of using options corresponding to a best fit allocation strategy. Looking at data from previous experiments [5, 11] we have found that having a few very large areas of free clusters at the end of a NTFS partition is the standard situation, thus the allocation strategy used by Windows together with NTFS is only best fit in special circumstances. Likewise the allocation strategy is not strictly best fit when dealing with stream written files, where the allocation algorithm is creating increasingly larger fragments as more data is written (the fragment size and the currently written size correlates). However, the fragments are allocated to the best fitting free areas, so there is a foundation of best fit behavior in the algorithm.

We have not found any literature empirically studying the inner workings of the allocation strategy used in Windows 7 and 10 partitions formatted as NTFS. Therefore already the existence of this work contributes to the digital forensics field. The result can furthermore be used to verify timestamps, rebuild files in file carving and determine the type of file on a high level by looking at how the sizes

of the allocated areas increases or decreases. Block written files are allocated in decreasing order of fragment size and stream written files are allocated in increasing order of fragment size. We also found that very small allocated areas (1 to 5 clusters) belong to the start of stream written files with high probability.

As future work we will expand the experiment to be able to isolate the different parameters affecting the behavior of the allocation algorithm. We also need more data to further strengthen our results and conclusions. It would also be of great interest to include other OSs and file systems than Windows and NTFS in the experiment.

Funding Information Open access funding provided by Swedish Defence Research Agency.

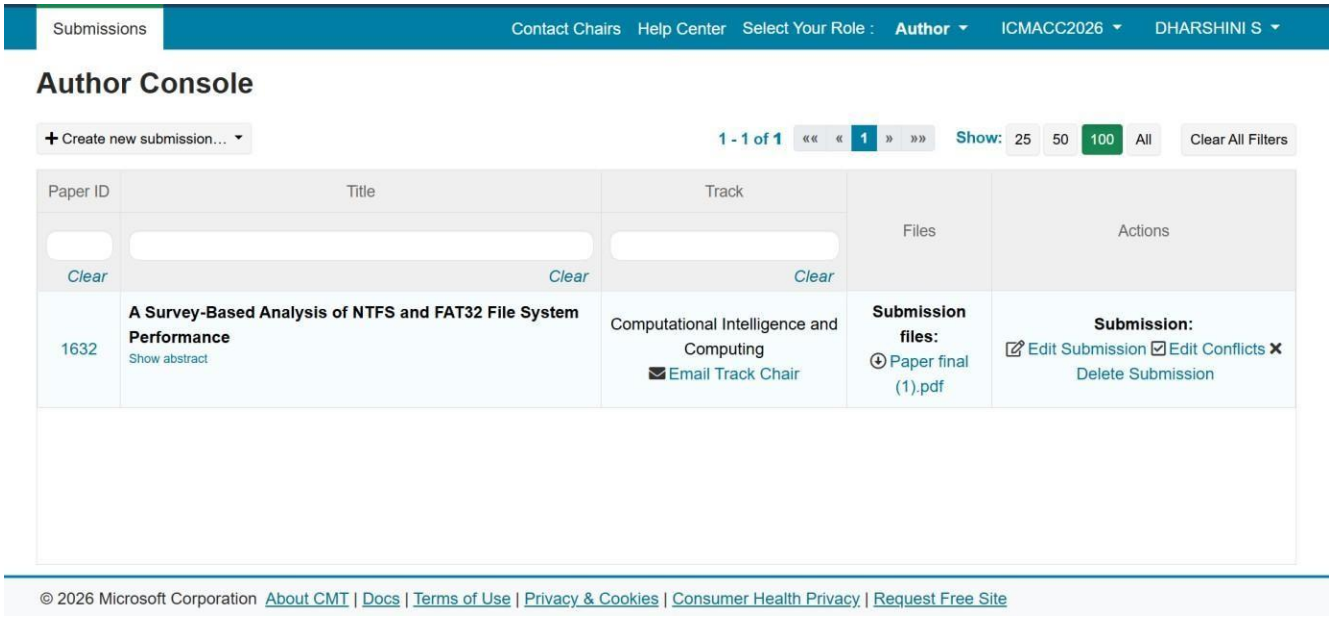
Open Access This article is distributed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution, and reproduction in any medium, provided you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license, and indicate if changes were made.

References

1. Carrier B (2005) File system forensic analysis. Addison-Wesley Professional, Boston
2. Carrier B (2014) Tsk tool overview. http://wiki.sleuthkit.org/index.php?title=TSK_Tool_Overview
3. Hughes J (2009) The four stages of NTFS file growth. https://blogs.technet.microsoft.com/askcore/2009/10/16/the-four-stages-of-ntfs-file-growth/. Accessed 24-10-2018
4. Karresand M (2008) Completing the picture — fragments and back again. Licentiate thesis, Linköping Institute of Technology, Linköping University, Sweden
5. Karresand M, Axelsson S, Dyrkolbotn GO (2019) Using NTFS cluster allocation behavior to find the location of user data. Digital Investigation. 29(Supplement):S51–S60
6. Karresand M, Shahmehri N (2006) File type identification of data fragments by their binary structure. In: Proceedings from the seventh annual IEEE systems, man and cybernetics (SMC) information assurance workshop, 2006. IEEE, Piscataway, pp 140–147
7. Karresand M, Shahmehri N (2006) Oscar – file type and camera identification using the structure of binary data fragments. In: Haggerty J., Merabti M. (eds) Proceedings of the 1st conference on advances in computer security and forensics, ACSF. The School of Computing and Mathematical Sciences, John Moores University, Liverpool, pp 11–20
8. Karresand M, Shahmehri N (2006) Oscar – file type identification of binary data in disk clusters and RAM pages. In: Proceedings of IFIP international information security conference: security and privacy in dynamic environments (SEC2006), Lecture notes in computer science, pp 413–424
9. Karresand M, Shahmehri N (2007) Oscar – using byte pairs to find file type and camera make of data fragments. In: Blyth A., Sutherland I. (eds) Proceedings of the 2nd European conference on computer network defence, in conjunction with the first workshop on digital forensics and incident analysis (EC2ND 2006). Springer, Berlin, pp 85–94
10. Karresand M, Shahmehri N (2008) Reassembly of fragmented jpeg images containing restart markers. In: Proceedings - 4th annual European conference on computer network defense, EC2ND 2008, pp 25–32
11. Karresand M, Warnqvist Å, Lindahl D, Axelsson S, Dyrkolbotn GO (2019). In: Advances in Digital Forensics XIV, chap. 8 Creating a map of user data in NTFS to improve file carving, pp. 133–158. Springer International Publishing AG, Cham
12. Microsoft: System requirements (2017). https://support.microsoft.com/en-gb/help/12660/windows-8-system-requirements. Accessed 30-04-2018
13. Microsoft: Windows 10 system requirements (2017). https://support.microsoft.com/en-us/help/4028142/windows-windows-10-system-requirements. Accessed 30-04-2018
14. Microsoft: Windows 7 system requirements (2017). https://support.microsoft.com/en-us/help/10737/windows-7-system-requirements. Accessed 30-04-2018
15. Microsoft: How ntfs works (2018). https://technet.microsoft.com/pt-pt/library/cc781134(v=ws.10).aspx. Accessed 30-09-2018
16. Minnaard W (2014) The Linux FAT32 allocator and file creation order reconstruction. Digital Investigation 11(3):224–233. https://doi.org/10.1016/j.diin.2014.06.008. Special Issue: Embedded Forensics
17. Pal A, Memon N (2009) The evolution of file carving. IEEE Signal Proc Mag 26(2):59–71. https://doi.org/10.1109/MSP.2008.931081
18. Poisel R, Tjoa S (2013) A comprehensive literature review of file carving. In: 2013 International conference on availability, reliability and security, pp 475–484. https://doi.org/10.1109/ARES.2013.62
19. Roussev V, Garfinkel S (2009) File fragment classification-the case for specialized approaches. In: 2009 Fourth international IEEE workshop on systematic approaches to digital forensic engineering, pp 3–14. https://doi.org/10.1109/SADFE.2009.21
20. Silberschatz A, Galvin P, Gagne G (2012) Operating system concepts, 9th edn. Wiley, Hoboken
21. Stallings W (2012) Operating systems – internals and design principles, 7th edn. Pearson Education Inc., Upper Saddle River
22. (2017) Superuser: What block allocation algorithm does ntfs use?. https://superuser.com/questions/274855/what-block-allocation-algorithm-does-ntfs-use. Accessed 24-01-2019
23. Tanenbaum A, Bos H (2015) Modern operating systems, 4th edn. Pearson Education Inc., Upper Saddle River
24. Tse W (2011) Forensic analysis using fat32 file cluster allocation patterns. Master's thesis, University of Hong Kong
25. Willassen S (2008) Finding evidence of antedating in digital investigations. In: 2008 Third international conference on availability, reliability and security, pp 26–32. https://doi.org/10.1109/ARES.2008.149
26. Willassen S (2008) Methods for enhancement of timestamp evidence in digital investigations. Ph.D. thesis, Norwegian University of Science and Technology, Faculty of Information Technology, Mathematics and Electrical Engineering, Department of Telematics

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

ANNEXURE III



PAPER DETAILS

- **Conference:** ICMACC2026
- **Paper ID:** 1632
- **Title:** A Survey-Based Analysis of NTFS and FAT32 File System
- **Track:** Computational Intelligence and Computing
- **Submission Status:** Submitted and Under Review
- **Authors:** [DEEPA K] [DHANSHIKA R] [DHARSHINI S] [LITHIGAA GJ]
- **Submission Date:** April 2026
- **Submission Platform:** IEEE Author Console (CMT)

ANNEXURE IV

PLAGIARISM REPORT

plag check.pdf

Sri Ramakrishna Engineering College

Document Details

Submission ID
trn:oid:::3117:579632998

Submission Date
Apr 17, 2026, 9:03 AM GMT+5:30

Download Date
Apr 17, 2026, 9:04 AM GMT+5:30

File Name
plag check.pdf

File Size
283.6 KB

37 Pages

6,674 Words

37,677 Characters

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

37 Not Cited or Quoted 5%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

4 Missing Citation 1%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

5% Internet sources
3% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.